VOSS: Scheduling Attested Session State for Serverless Computing on Volunteer Machines

Aalhad Sawane

***Abstract*—Short serverless functions are dominated by cold starts and network distance, while capacity close to users sits idle on consumer machines. Serving latency-sensitive invocations from such volunteer machines is hard for two reasons: owners end sessions abruptly, which destroys every warm sandbox on the host, and owners are untrusted and can read or tamper with tenant code and data. Volunteer-computing systems cope through redundant execution and long deadlines, and serverless schedulers assume trusted, always-on fleets. We observe that remote attestation of a locked, measured appliance image reduces a malicious owner who controls software, power and network to omission and timing failures. What remains is a scheduling problem over *attested session state*: tenant keys provisioned on a host and warm sandboxes are expensive to establish, are lost when the session ends, and every additional key holder enlarges the tenant's attack surface. We present VOSS, a scheduler built around this problem. A slow control loop plans where keys and warm state live, with coverage that stays feasible when any f hosts defect; a fast loop places each invocation with a cost model and hedges risky ones. Failure detection uses end-to-end progress instead of liveness, which exposes gray failures; scheduler failover uses leases and epoch fencing without consensus among volunteers; and attested receipts make accounting verifiable. In a deterministic simulator calibrated on a TPM-equipped machine, VOSS meets a 3 s sync P99 target in 9 of 9 cells (churn by adversary fraction) and 13 of 16 (substrate by load), where the strongest simple baseline never does. Planning lowers P99 by 54–59% against no planning, and progress-based detection keeps success at 99.95% with 20% omitting owners where heartbeat-only detection falls to 96.8%. Several designed components do not pay off in our experiments (geo-clustering, reputation, and tiered caching), an exact solver is not distinguishable from a greedy heuristic at our sample size, and a commercial-style reference remains faster; we report these with all other hypotheses.**

***Index Terms*—Serverless computing, volunteer computing, remote attestation, trusted execution, scheduling, gray failure, tail latency.**

I. INTRODUCTION

FUNCTION-AS-A-SERVICE (FaaS) platforms run short, event-driven functions on demand. In production traces most functions finish in well under a second [1], so their end-to-end latency is set not by computation but by the *cold start* (creating a sandbox and loading the language runtime) and by network distance to the host [2]–[4]. Providers meet latency targets by keeping sandboxes warm in datacenters, which pins memory around the clock [1], [5]. Meanwhile, consumer desktops and laptops are numerous, geographically close to users, and idle most of the day. Volunteer computing has harvested such machines for two decades [6], [7], but only for batch work whose deadlines are measured in hours or days.

[TODO: author list, affiliations, funding and acknowledgments; this draft names only the repository owner.]

Serving *latency-sensitive* invocations from volunteer machines raises two problems that batch systems avoid. **Churn** is the first. An owner can power off, reboot or reclaim the machine at any moment, and all state the platform has built on the host (warm sandboxes, cached snapshots, provisioned secrets) disappears with the session. For a function that runs for tens of milliseconds, rebuilding that state is the dominant cost. **Distrust** is the second. The owner controls the hardware and operating system and can read tenant code and data, forge results, or defect at the worst moment. The two problems interact: the usual defense against distrust is to establish trust in a host before using it, and churn destroys that trust each time a session ends.

A. Why existing approaches do not solve this

Each line of prior work handles part of the problem (Section III gives the full comparison). *Volunteer computing* tolerates untrusted hosts by redundant execution and result voting [7], which multiplies cost by the replication factor, requires deterministic side-effect-free tasks, and cannot hide inputs from the host. *Serverless schedulers and keep-alive policies* minimize cold starts [1], [5] but assume trusted, always-on servers, so neither churn nor the cost of establishing trust appears in their models. *Confidential serverless designs* protect tenants from the platform operator with trusted execution environments [8], [9] but target provider-managed fleets and treat attestation as a one-time deployment step rather than a resource with a cost, a lifetime and a security price. *Heartbeat-based failure detection* cannot see a host that keeps reporting liveness while discarding work, a *gray failure* [10] that a malicious owner can create at will.

B. Our insight and approach

Our starting point is that attestation changes the fault model. If tenant code and the scheduler's own code run only inside a measured, owner-locked appliance image whose TPM-backed evidence a verifier checks, then a hostile owner can no longer read, forge or tamper through software. What remains is an *omission or timing failure* in the dependability taxonomy [11], [12]: the host powers off, drops, delays or throttles. This is the failure class that churn already forces a volunteer platform to tolerate, so one set of mechanisms covers crashes, churn and malice [13], [14].

The reduction exposes the actual scheduling problem. A host is *attested* for the duration of a session, becomes *provisioned* for a tenant once the tenant's key broker has released a secret to it, and holds *warm* state for a function once a sandbox or snapshot is ready. This attested session state has three properties

that conventional cluster schedulers do not model: it is *costly to establish* (on our measurements, a powered-off machine takes 65 s to become an attested worker, three to four orders of magnitude above restoring a snapshot or resuming a warm sandbox); it is *perishable*, lost at the end of every session; and it is *exposing*, since each host that holds a tenant's key widens that tenant's attack surface. Placing an invocation on a host that is not yet provisioned is therefore a decision with a cost that the scheduler must weigh, and so is asking another host to hold the key.

We present VOSS, a scheduler for volunteer serverless computing built on this view (Fig. 2). VOSS (i) separates *where state lives* from *where an invocation runs* with a two-timescale design: a slow planner decides which hosts are provisioned and warm for each tenant under a robustness constraint, and a fast placer assigns each invocation by minimising its own expected SLO-violation probability, weighed together with latency and key exposure (Section VI-B), and hedges risky ones; (ii) detects failures from end-to-end *progress* rather than liveness and replaces failed schedulers with lease and epoch fencing, without consensus among volunteers; and (iii) makes accounting verifiable through attested receipts instead of a blockchain.

C. Contributions

1) **Problem formulation.** We reduce malicious owners to omission and timing failures through attestation and formulate volunteer FaaS scheduling as the management of attested session state with explicit establishment cost, perishability and exposure (Sections II, V).
2) **Two-timescale scheduling.** A robust provisioning planner with cardinality-constrained coverage plus a cost-based placer with risk-proportional hedging (Section VI). We find no detectable advantage of the exact mixed-integer solution over a greedy planner at our sample size.
3) **Fault tolerance without volunteer consensus.** Progress-based detection of gray failures, lease and epoch-fenced scheduler failover on soft state, and attested receipts for accounting (Section VII).
4) **A calibrated, reproducible evaluation.** A deterministic simulator with parameters measured on a TPM-equipped machine, an operating-regime analysis that reports where VOSS and where simple baselines win, and hypothesis tests with pre-specified decision rules (Sections VIII, IX).

D. Summary of findings

Managing attested session state is the largest lever: planning lowers sync P99 by 54–59% against no planning and 27–57% against keep-alive under medium and high churn. Progress-based detection is decisive against omitting owners. VOSS meets a 3 s sync P99 target in most cells of two regime maps where the strongest simple baseline never does, but the simple baseline is the only feasible system on confidential-VM substrates at high load. Several designed components do not pay off: geo-clustering is slower than a single scheduler at our scale, the exact solver's advantage over greedy planning (8.1% at high churn) is not distinguishable from zero, reputation changes P99 by 2.4%, and a commercial-style reference has lower latency. All results are from simulation and a single calibration machine (Section X-B).

II. Background and Problem Statement

Terms. A *tenant* is a customer of the platform: the developer or organization that writes functions and deploys them. A *function* is one piece of tenant code; an *invocation* is one call of it. The *provider* operates the platform (VOSS), and a *volunteer* or *owner* owns a machine that donates capacity. A tenant's *secret* is a cryptographic key that only the tenant (through its key broker) controls; it decrypts the tenant's code bundles and the snapshots of the tenant's sandboxes, which hold decrypted memory. A host that holds the key can therefore read that tenant's code and the data its functions handle in memory. A *sandbox* is a microVM that runs one function, and a sandbox is *warm* when it is already started and idle, ready to take an invocation.

A. Latency of short functions

An invocation's end-to-end latency decomposes as

$$T = T_{\text{route}} + T_{\text{decide}} + T_{\text{queue}} + T_{\text{prov}} + T_{\text{start}} + T_{\text{net}} + T_{\text{exec}} + T_{\text{retry}}, \tag{1}$$

the time to choose a region and a host, wait for a free slot, make the host able to run the tenant's code (T_{prov}, the provisioning of secrets, zero if already done), bring a sandbox to the point of executing the function (T_{start}), move input and output, run, and repeat work lost to failures. For functions of tens to hundreds of milliseconds, T_{start} dominates unless a sandbox is already warm. Platforms therefore keep sandboxes alive after use or restore them from snapshots [1], [4], [5], [15]. We distinguish four *start-up tiers* for a function on a host: **L0**, nothing cached (fetch, verify, boot, initialize); **L1**, code and root file system on local disk (boot and initialize); **L2**, a snapshot on local disk (restore); **L3**, an idle warm sandbox in memory (dispatch). Higher tiers cost memory or disk, so deciding *which* functions to keep at *which* tier on *which* hosts is itself a scheduling problem.

Volunteer hosts add a term that datacenters do not have. Before a host may run tenant code at all it must be attested, and before it may decrypt a tenant's code and data it must hold that tenant's key. Fig. 1 places these costs on one scale using measurements on a consumer laptop with a TPM (Section VIII-B). Resuming a warm sandbox is milliseconds, booting a microVM with a warm page cache about 154 ms and with a cold cache 1.45 s. Reaching an attested worker from a powered-off machine takes 65 s (median of 9 boots): 21 s for the machine's firmware, boot loader and kernel start, then 44 s from kernel start until the verifier accepts a TPM quote. These are upper bounds, because the appliance was exercised in host mode on a hard disk (Section VIII-B). Establishing trust is thus three to four orders of magnitude more expensive than restoring or resuming a sandbox, which is why *where trust is established* cannot be left to the per-request path.

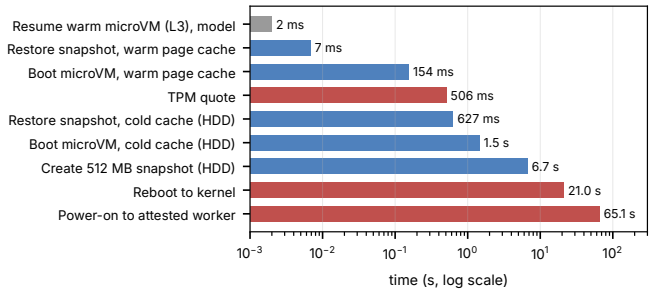


Fig. 1. Time to reach each state on the calibration machine (log scale). Blue: sandbox start-up; red: establishing an attested worker; grey: model value (not measured). *Warm page cache*: the snapshot or image file is already in memory; *cold*: it is read from the hard disk. *Reboot to kernel* is the physical machine's firmware, boot loader and kernel start, not a microVM boot. Medians over repeated runs (Section VIII-B).

B. Remote attestation and secret provisioning

We use the vocabulary of the IETF RATS architecture [16]. The *attester* is the volunteer host. During *measured boot*, each boot stage (firmware, boot loader, kernel image) is hashed into a platform configuration register (PCR) of the TPM, so the register values are a hash chain of what ran [17]. On request the TPM signs selected PCR values together with a verifier-chosen nonce using an attestation key; this signed object is the *quote*, and the attestation key is certified by the TPM's endorsement key, whose certificate chains to the manufacturer. The *verifier* checks the signature and the certificate chain, replays the event log against *reference values* for known-good software, and issues an *attestation result*. A *relying party*, here the tenant's *key broker service* (KBS), releases a secret only to an attester with an acceptable result [18]. Two properties matter for scheduling. A quote shows what software booted at one moment, so evidence goes stale and must be refreshed. And attestation says nothing about *availability*: a host that passes attestation can still stop working.

C. Volunteer hosts and failure model

A volunteer host alternates between off and a *session*: an interval during which it is booted into the platform. Session lengths are heavy-tailed and depend on the time of day [19]. A session ends gracefully with advance notice or abruptly (reboot, power loss), sometimes for a whole city at once. We use the standard classification of failures into crash, omission, timing and Byzantine [11], [12]. An *omission failure* is a host that does not send a result it owes; a *timing failure* sends it too late. Churn produces crashes; an adversarial owner who cannot forge or tamper can still choose *when* to omit or delay, selectively and while keeping liveness signals healthy. Such *gray failures*, where a component appears healthy to simple probes but fails applications [10], defeat heartbeat-based detectors, including accrual detectors [20].

D. Problem statement and goals

Input. A stream of invocations, each with a tenant, function, service class (synchronous, asynchronous, or preemptible filler), deadline, input size and an idempotence flag (calling the function twice has the same effect as calling it once; the platform retries and hedges only such functions, and for the others a lost result is a failure), arriving from geo-distributed clients; and a dynamic population of volunteer hosts, each with slots, memory and disk budgets, a location and failure domain, a trust substrate, and a session that can end at any time. **Decisions.** Which hosts to attest-and-provision for which tenants, which functions to keep at which start-up tier on which hosts, and which host runs each invocation (possibly hedged by a duplicate). **Objectives.** Meet each class's latency SLO (success rate, deadline-met fraction, P99 of synchronous invocations), while limiting donor resources (memory held warm), *key exposure* (Section V-D), and wasted work.

We pursue four goals.

- G1 *Confidentiality and integrity.* A malicious owner with software-level control learns no tenant secret and forges no result (threat model of Section V-A).
- G2 *Latency under churn.* Tail latency stays within the SLO although sessions end at any time.
- G3 *Liveness under omission.* A host that accepts work and discards it, or a scheduler that stalls, is detected and bypassed within seconds, without trusting liveness signals.
- G4 *Verifiable accounting.* Contributed work is recorded so that a host cannot claim work it did not do.

Non-goals. We do not defend against physical attacks on the TPM, vulnerabilities in the appliance itself, side channels, or failure of the provider-operated control plane (Section V-A). We treat invocations as independent; function workflows (DAGs) are discussed in Section X-A.

III. Related Work: Why Existing Approaches Fall Short

Table I summarizes how five lines of work address the properties VOSS needs. No line covers them together; the paragraphs below state what each does and why it is insufficient here.

Volunteer computing. BOINC-style systems harvest idle machines for scientific batch work [6], [7]. Because hosts are untrusted, they replicate tasks and accept a result when a quorum agrees, and they use deadlines to tolerate hosts that disappear. Replication multiplies cost, presumes deterministic tasks without side effects, protects integrity but not the confidentiality of inputs, and works on a time scale of hours. None of this fits invocations that take milliseconds, may be non-deterministic, and carry private data. VOSS instead removes the integrity and confidentiality concern with attestation, so redundancy is needed only against omission, and then only for risky invocations.

Serverless scheduling and keep-alive. The cold-start literature decides when to keep sandboxes alive or pre-warm them [1], treats keep-alive as caching [5], or accelerates start-up with lighter sandboxes and snapshots [2]–[4], [15]. Low-latency schedulers place short tasks by sampling [21] and large-scale cluster managers pack tasks onto fleets [23]. All assume trusted, effectively always-on machines. A warm sandbox is lost only

TABLE I
COVERAGE OF THE REQUIRED PROPERTIES BY PRIOR APPROACHES (● ADDRESSED, ○ PARTLY, BLANK: NOT ADDRESSED). THE ASSESSMENT IS OURS, FROM EACH APPROACH'S STATED GOALS.

	Untrusted hosts	*Host churn*	*Latency SLOs*	*Plans trust and warm state*	*Gray-failure detection*	*Verifiable accounting*
Volunteer computing [7]	●	●			○	○
Serverless schedulers [1], [5], [21]			●	○		
Harvested-resource serverless [22]		●	●	○		
Confidential serverless [8], [9]	●		●			
Trusted-component fault tolerance [13], [14]	●					
VOSS	●	●	●	●	●	●

through a policy decision, never because its owner closed the lid, and the model has no resource corresponding to a provisioned secret. VOSS keeps the start-up-tier vocabulary but adds a per-session trust step and plans warm state under churn and exposure constraints.

Harvested and edge serverless. Serverless platforms on harvested datacenter VMs handle evictions of the underlying resources [22], and edge schedulers place containers near data and users [24]. Their hosts are operated by the provider or a trusted party, so the failure model is eviction rather than silent omission by an adversary, and no secret is entrusted to a party that might read it.

Confidential serverless. Trusted execution environments protect containers and functions from the host operating system and the platform operator [8], [25], [26], and recent work targets serverless functions in confidential VMs and containers [9], [27], and measurement studies quantify the cost of confidential VMs [28]. These designs address confidentiality on provider-managed server hardware. They do not address hosts whose sessions end unpredictably, and they do not treat the establishment of trust as a scheduled resource with a cost, lifetime and exposure. VOSS is substrate-agnostic (Section V-E): the same scheduler runs over a TPM-attested appliance on consumer hardware or over confidential VMs, with the substrate profile supplying establishment costs and assurance.

Trusted components and failure detection. Small trusted components (a TPM-like counter or log) restrict what a faulty node can do and reduce replication needed for Byzantine agreement [13], [14]. We use the same idea differently: attestation does not make agreement cheaper; it makes the faulty node *non-Byzantine*, which lets us replace agreement among volunteers by a provider-run arbiter (Section VII-C). For detection, heartbeat and accrual detectors [20] infer liveness, which is not progress; gray failures [10] motivate observing end-to-end progress, which VOSS does at the trusted router and the attested scheduler.

Hedging and robust provisioning. Hedged requests reduce tail latency by duplicating slow work [29]. VOSS applies hedging *in proportion to estimated risk*, on a different failure domain, and bounds the extra work by a budget. For the planner we use cardinality-constrained (budgeted) robust optimization [30], which fits the question "is demand still covered if any f hosts defect together?" without assuming a probability model for adversaries.

IV. SYSTEM OVERVIEW

A. Design principles

Three ideas organize the design. **(P1) Attestation turns distrust into unreliability.** Everything that touches tenant secrets or makes scheduling decisions runs inside the attested appliance, so the rest of the system only has to tolerate omission and delay (Section V). **(P2) Trust is a managed resource.** Attestation, key provisioning and warm state have costs, lifetimes and an exposure price, and they are planned on a slower timescale than invocation placement (Section VI). **(P3) Liveness is a hint; progress is evidence.** Signals from untrusted hosts, including heartbeats (periodic "I am alive" messages), drain notices (an owner's announcement that they will leave by a deadline) and self-reported health, are never sufficient. Failures are detected where work is observed by a trusted or attested party (Section VII).

B. Architecture and roles

Fig. 2 shows the components and Table II states what each is trusted for. Four kinds of parties take part.

Provider control plane. A provider-operated tier is assumed reliable and is trusted for *availability and accounting*. It is never given tenant keys; whether it must also be trusted for confidentiality depends on the tenant's key-release policy (Section V-C). *Regional routers* receive invocations, choose a cluster, keep a pending copy until a worker accepts, observe progress, and write the receipts that are the ground truth of work. The *orchestrator* partitions workers into geographic clusters of tens of hosts (which bounds the planner's problem size and intra-cluster latency), appoints one scheduler per cluster, and rebalances clusters. The *verifier and registrar* appraises attestation evidence and issues session certificates. An *operational database* private to this tier stores identities, sessions, cluster epochs, checkpoints and receipts, and an *artifact registry* serves encrypted code bundles and root file systems.

Tenants. Each tenant operates, or delegates to a service it trusts, a *key broker service* (KBS) that holds the tenant's secrets and releases them only to hosts with an acceptable attestation result (Section V-C). The provider does not hold tenant keys, which is why the key broker is not provider-operated. Neither the scheduler nor the control plane ever sees a tenant key.

Volunteer hosts. A volunteer machine reboots into the *attested appliance*, a locked, measured operating-system image containing the worker agent, the Firecracker microVM monitor [2] and the scheduler code. The *worker* runs each invocation in its own microVM and keeps a content-addressed, encrypted cache of code and snapshots. The *scheduler* is a role: the same image runs on every worker, so any worker can be promoted without booting anything, and one worker per cluster

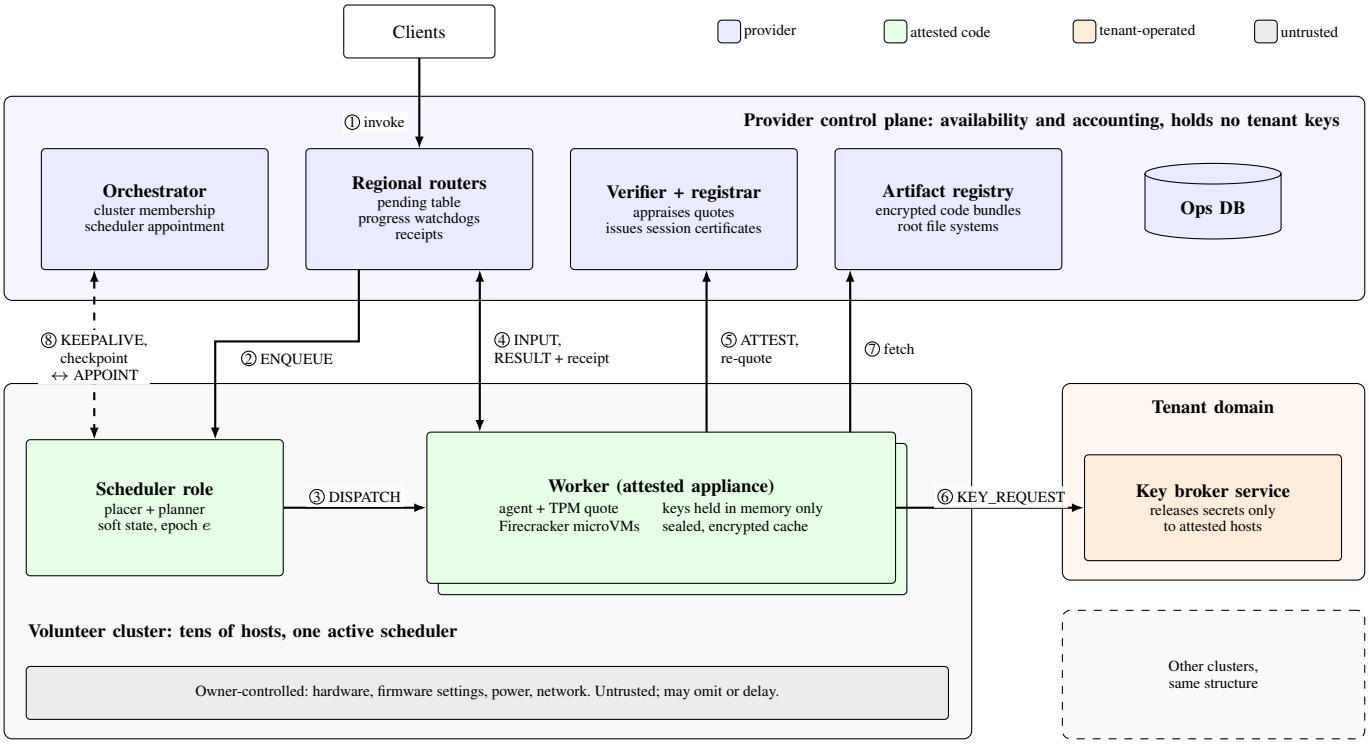


Fig. 2. VOSS architecture and trust boundaries. Shading shows who is trusted for what: the provider control plane for availability and accounting (it is never given tenant keys), attested code on owner-controlled hardware, the tenant-operated key broker, and the owner's untrusted hardware. Numbered flows follow one invocation (Fig. 3); the dashed flow is control traffic off the hot path.

TABLE II
ROLES, LOCATION, AND WHAT EACH IS TRUSTED FOR.

Role	Runs on	Trusted for	Not trusted for
Router	provider	availability, accounting	tenant secrets (never receives them)
Orchestrator, verifier, DB, registry	provider	availability, accounting, appraisal	tenant secrets (but see Section V-C)
Tenant KBS	tenant	releasing secrets per policy	availability of volunteer hosts
Scheduler	role on an attested worker	correct placement (attested code)	availability: its host may omit or delay
Worker	volunteer host, attested appliance	executing code faithfully	availability; anything outside the attested stack

is appointed to perform the scheduling functions of Section VI. While the appliance runs, the owner cannot use the machine: donation is a scheduled or dedicated session, not background use of an idle desktop. The host itself (power, network, firmware settings) remains under the owner's control, which is why the scheduler's *host* can still omit or delay even though its *code* cannot misbehave.

C. Life of an invocation

Fig. 3 follows one synchronous invocation. A router picks a cluster by round-trip time, health and provisioned capacity for the tenant, and sends only *metadata* to that cluster's scheduler. The scheduler chooses a worker (and possibly a hedge), dispatches it, and tells the router who was assigned; the router streams the payload to the worker over a TLS session terminated inside the attested appliance (an *attested* session: the worker authenticates with the certificate that the verifier issued after checking its quote, Fig. 5). The scheduler therefore never touches payloads, results or tenant keys. If the worker is not yet provisioned for the tenant it requests the key from the tenant's KBS, then climbs the start-up tiers (Section IV-D) and runs the function. The result returns directly to the router together with a receipt signed by the worker's session key (Section VII-F); the latency visible to the client ends there. Every term of Eq. (1) is logged separately per invocation, which is what lets Section IX attribute latency to its causes.

D. Sessions and start-up tiers

A worker alternates between OFF and a *session* (Fig. 4a). It becomes schedulable only after booting and being attested, which on our calibration machine takes 65 s from power-on. A session ends abruptly (reboot, power loss, crash) or gracefully (the owner announces that they will leave by a deadline, a *drain*). Session end discards every secret and warm sandbox, but disk caches remain as ciphertext and are valid again after the next attestation. The disk caches hold encrypted code bundles, root file systems and snapshots; a cached item is evicted by a value-density rule (a GreedyDual-size-frequency variant) with LRU as the baseline (Section IX-F).

Within a session a function on a host sits at one of the start-up tiers of Section II (Fig. 4b), and a separate flag $K[t, j]$ records whether the host holds tenant t's key. Start-up costs depend on the tier (warm microVM boot 154 ms with a warm page cache, 1.45 s with a cold one; snapshot restore 7 ms warm,

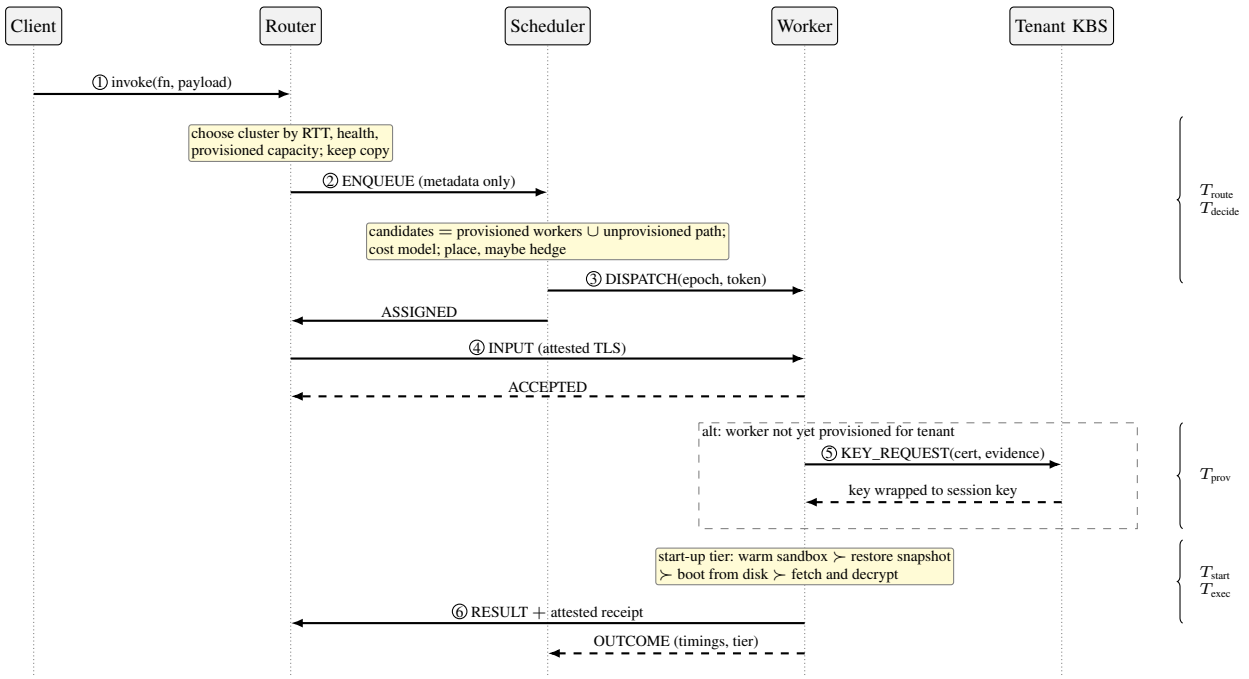


Fig. 3. Life of a synchronous invocation. The scheduler handles only metadata. The shaded alternative is the path through an unprovisioned worker, which the planner tries to keep off the critical path. Braces map messages to the terms of Eq. (1) (T_{queue} and T_{net} occur between steps 4 and 6).

0.63 s cold; Fig. 1) and on whether $K[t, j] = 1$. These two facts, a per-session trust step and a per-tenant key flag, are what the generic lifecycle engine in our simulator interprets from data for every substrate (Section V-E).

E. State management and communication rules

Six rules keep the hot path fast and failures recoverable. (i) *All cross-node interaction is a typed message* (a message with a fixed schema such as ENQUEUE or DISPATCH, sent over a connection); components never call each other's code. Workers sit behind NAT, so each opens outbound persistent TLS streams to its scheduler, the verifier, and routers, and pushes ride those streams. (ii) *The operational database is private to the control plane*, and workers and schedulers reach it only through messages. (iii) *The invocation hot path never touches the database*; the hot path is the route one invocation takes from the router through the scheduler to a worker and back, and a database round trip would land in latency. (iv) *Live state stays where it is born*: queues, caches and keys in workers, undispatched work in routers, cost matrices in the scheduler. After a scheduler change it is recovered by asking the workers, not by reading it back from the database (Section VII-C). (v) *Routers write the ground truth of work*, as attested receipts (a record of each executed invocation, signed by the worker's session key, Section VII-F). (vi) *Every channel terminates inside an attested party*: the appliance, the control plane, or the KBS. The session certificate issued after attestation is the credential for all of them.

F. Component contracts

Table III states for each decision-making component what it consumes, what it produces, what it must respect and what it is allowed to know, so that every behaviour in the following sections can be traced to an input and an owner. Parameters are given in Table VIII (scheduling), Table XIII (fault tolerance) and the calibration bundle (costs).

V. Trust Model and Establishment

This section states the threat model, shows how attestation reduces malicious owners to omission and timing failures, and defines the three pieces of attested session state that the scheduler manages: the attested session, tenant key provisioning, and the exposure they create.

A. Threat model

Table IV summarizes the adversaries. We trust the CPU, its TPM and memory encryption; the measured appliance image (kernel, monitor, agent, scheduler code); each tenant's own key broker; and the provider-operated control plane, for availability and accounting only. Out of scope are laboratory-grade physical attacks such as voltage glitching of firmware TPMs [31], vulnerabilities in the appliance itself, side channels and traffic analysis, and attacks on or failure of the control plane. Property 1 below concerns an owner who controls the machine's software, power and network. An owner with physical access who can read memory while the appliance runs (cold-boot, DMA or bus probing) is outside it unless the host encrypts memory; the assurance score a_j (Section V-D) expresses this residual only as a policy number.

Property 1 (threat reduction). Under these assumptions, every behavior of a worker or scheduler host that other components can observe, and that is reachable by software, is a crash, an omission or a delay. The argument is that the owner can neither run code other than the measured image (it

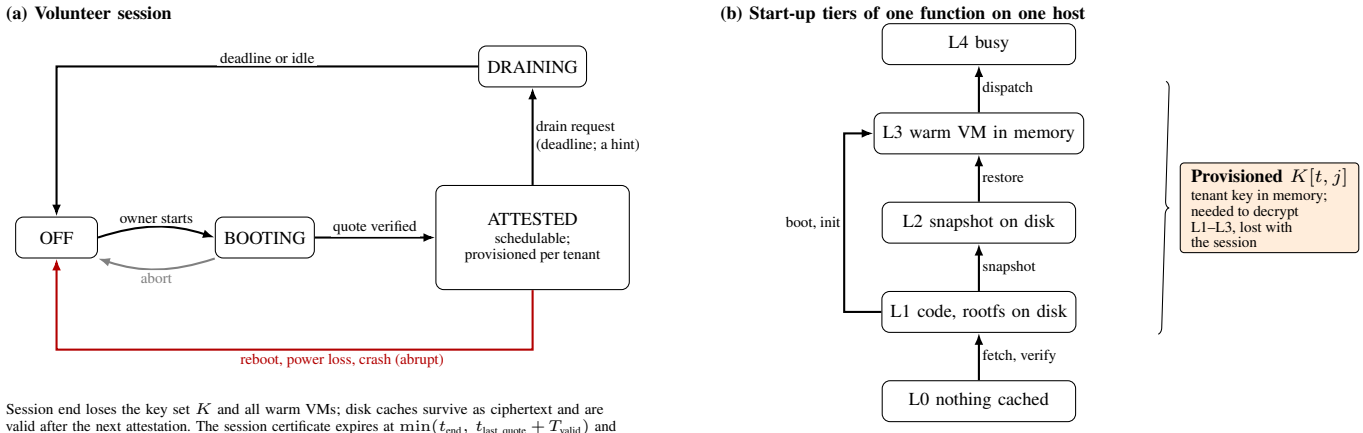


Fig. 4. (a) A volunteer session. Trust is established once per session (booting, attestation) and lost when it ends. (b) Start-up tiers of one function on one host. Moving up costs time and memory or disk; the tenant key flag $K[t, j]$ is separate from the tier: the tier says what is cached, and $K[t, j]$ says whether the host can decrypt it (L1 and L2 files stay on disk as ciphertext when the key is lost with the session).

TABLE III
CONTRACTS OF THE DECISION-MAKING COMPONENTS. "KNOWS" LISTS THE INFORMATION CLASS (TABLE IX).

Component	Input	Output	Constraints	Knows
Router (regional)	invocation, cluster summaries each 1 s	cluster choice, spill, receipt rows	dispatch watchdog 1 s; sync spill after 1.5 s	O, measured cold prices
Orchestrator	scheduler keep-alives and summaries, worker joins and leaves, stall reports	cluster membership, scheduler appointments (epoch CAS), revocations	single writer per datum; epoch fencing; reliable by assumption	O
Scheduler: planner	view, forecasts, tenant policy	plan $\{y, s, w, c\}$, actions	robust coverage, budgets, domain caps, assurance floor	O, H, F, C, P
Scheduler: placer	invocation metadata, view, plan	host, hedge, reject or defer	feasibility, hedge budget, admission rule	O, H, C, P
Verifier	quote, event log, EK chain	session certificate (assurance, expiry) or rejection	reference measurements, validity period	measured-boot evidence
Tenant KBS	certificate, policy	wrapped key or denial	tenant policy only	tenant policy
Worker agent (attested)	dispatch, plan actions, input	result and receipt, heartbeat	envelope declared by owner; epoch fencing	local state

TABLE IV
ADVERSARIES, THE MECHANISM THAT NEUTRALIZES THEM, AND THE RESIDUAL CAPABILITY.

Adversary	Capability	Neutralized by	Residual
Volunteer owner	controls hardware, OS settings, power, network; may boot modified software, emulate a TPM, replay, delay, drop, create identities	measured boot and TPM attestation; vendor-certified endorsement key; owner-locked image with no login; disk sealed to PCRs; IOMMU	**omission and timing**: power off, drop, delay, throttle; lies about anything outside the attested stack
Network attacker	observes and alters traffic	TLS terminated inside attested parties; keys bound to attestation	traffic metadata
Other tenants	malicious function code	microVM isolation (Firecracker)	microarchitectural side channels

would fail attestation, so no session certificate or tenant key is issued) nor read or modify the image's memory or its sealed state, and that results and receipts are signed with a key that never leaves the appliance. The owner retains only what lies outside the attested stack: power, the network path, and the timing of when to stop answering. The consequence we use throughout is that the fault tolerance mechanisms needed for churn already cover malice (Section VII), and that redundancy is no longer required for integrity.

Why not other mechanisms. Memory encryption alone is not a trusted execution environment: the operating system still sees plaintext and nothing is attested. Replicated execution with voting triples cost and presumes comparable outputs. Zero-knowledge or other verifiable computation is orders of magnitude too slow for functions of milliseconds. Server confidential VMs have the strongest isolation but exist only on server CPUs; we model them as a substrate (Section V-E). A blockchain ledger would protect against no adversary in this model, because receipts signed by an attested key are already third-party verifiable and the only residual risk, a trusted control plane deleting records, is better met by a signed transparency log (future work).

B. Attested appliance and session

The appliance is a unified kernel image signed for Secure Boot with a read-only integrity-protected root (dm-verity), Firecracker, the worker agent and the scheduler code, with no owner login, kernel lockdown, IOMMU and integrity measurement (IMA) enabled. The data partition holding the caches is encrypted with a key sealed to the expected PCR values. A session begins when the owner chooses to donate and reboots into the image, and it ends on owner reboot or power loss (abrupt), on an owner drain request with a deadline (graceful), or on failure.

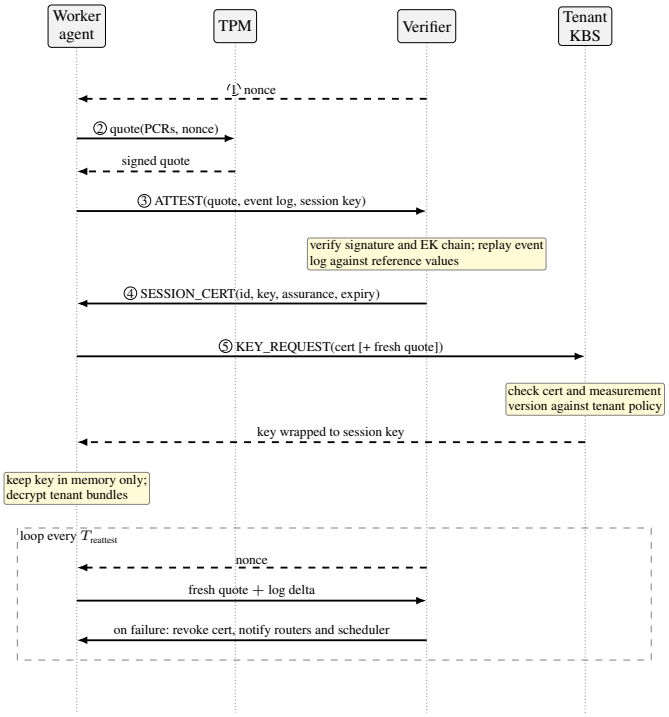


Fig. 5. Attestation and tenant key provisioning for one worker. The session key is bound into the quote, so the certificate is tied to this appliance instance.

Fig. 5 shows the attestation protocol, which follows Keylime-style remote attestation [18] in RATS roles (Section II-B). The agent generates a *session key pair* inside the appliance and binds its public half into the quote, as the hash of the key appended to the verifier's nonce. The verifier checks the quote signature and the endorsement-key certificate chain, replays the event log against reference values for the measured image, and, if the result is acceptable, issues a *session certificate* carrying the host identity, session public key, an assurance score and an expiry. Every channel of the system (to routers, the scheduler, the KBS, and peers) is TLS with this certificate, so there is no untrusted operating system on any path. Two properties are decisive for scheduling. *Trust is perishable*: the certificate expires at the earlier of session end and the last successful quote plus a validity period, is extended by periodic re-quotes, and is revoked on a failed re-quote, upon which the verifier notifies routers and the scheduler. And *identity costs a machine*: a worker's identity is the hash of its TPM endorsement key, so creating identities costs hardware [32].

C. Tenant key provisioning

Attestation proves that a host runs the appliance, not that it may see a given tenant's data. A tenant's KBS, operated by the tenant or by a service it trusts and not by the provider, holds a policy (for instance, accept certificates for appliance measurements at or above a version, optionally with a fresh quote). Making worker j *provisioned* for tenant t (the flag $K[t, j] = 1$) takes one round trip: the worker sends the KBS its certificate, the KBS checks the policy and returns the key wrapped to the session public key, and the worker keeps it in memory only. This round trip, plus decrypting the tenant's bundles, is the T_{prov} term of Eq. (1). Tenant code bundles in the registry and peer caches are encrypted with the tenant key, so any worker may store and forward ciphertext but only provisioned workers can use it. Snapshots contain decrypted memory, so they are written only by provisioned workers, shared only with provisioned workers of the same snapshot class (CPU model, kernel, monitor version), and counted in exposure.

Which party the tenant must trust for confidentiality depends on that policy. If the KBS accepts any certificate that the verifier issued for an approved measurement, then the verifier is trusted for confidentiality: a certificate issued to a non-genuine host would release the key. If the policy requires a fresh quote, the KBS checks the quote itself against its own reference values and does not depend on the verifier for confidentiality, at the price of one quote, which lies in T_{prov}. We assume that the verifier acts as the certificate authority for session certificates, with its root distributed in the appliance image and to the key brokers; this PKI is not evaluated, and the simulator charges a fixed calibrated cost for the policy check without distinguishing the two policies.

D. Exposure and assurance

Every provisioned host is a place from which a tenant's secret could leak if the trust assumptions fail. We define the *key exposure* of tenant t as

$$E_t = \sum_{j \in \mathrm{KH}(t)} (1 - a_j), \tag{2}$$

where $\mathrm{KH}(t)$ is the set of hosts holding t's key or decrypted artifacts and $a_j \in [0, 1]$ is the host's *assurance*: the product of a base value for its trust substrate and a firmware-policy factor (known-good versus trust-on-first-use), fixed when the verifier issues the session certificate. Staleness is not a continuous factor: a certificate simply expires, and a host whose quote would expire within the planning horizon is excluded from plans. The scheduler weights exposure by a tenant sensitivity σ_t and enforces an assurance floor for sensitive tenants. Exposure is a transparent proxy for attack surface, not a probability of compromise, and base assurances are policy parameters, not measured truths; we therefore report latency, exposure and robustness as a Pareto front and never as one weighted number.

Inputs, outputs and constraints. *Attestation* takes a TPM quote over the appliance's measured boot (event log, PCR values), the endorsement-key certificate chain and a fresh verifier nonce, and outputs a session certificate with assurance a_j and an expiry, or a rejection. It is constrained by reference measurements known to the verifier, the certificate validity period and re-quote interval, and the TPM's quote rate. *Provisioning* takes the certificate and the tenant's KBS policy and outputs the wrapped key or a denial. In the simulator both are sequences of simulated delays: network round trips through the geographic model, a TPM quote and verification service time from the substrate profile, and a fixed policy-check cost. The tenant key never reaches the scheduler or the orchestrator: the scheduler learns only that $K[t, j] = 1$, from the worker's heartbeat.

TABLE V
SUBSTRATE PROFILES. ASSURANCE BASES ARE POLICY PARAMETERS (SWEPT IN EXPERIMENTS); COSTS FOR THE APPLIANCE ARE MEASURED, THE OTHERS COME FROM THE LITERATURE.

Profile	Trust established per	Owner uses machine	Assurance base	Costs
`appliance_tpm`	session: reboot, measured boot, quote	no	0.80	measured
`cvm_snp`	confidential-VM session; key per function	yes	0.95	literature
`coco_pool`	confidential-container shell	yes	0.95	literature
`drtm_hv`	measured hypervisor launch	yes, with interference	0.75	assumed
`none`	–	yes	0.00	measured

TABLE VI
NOTATION FOR A CLUSTER OF ATTESTED WORKERS J.

t, f, j	tenant, function, worker; $t(f)$ is the tenant of f
H, T_{plan}	planning horizon, planning period
$y_{tj} \in \{0,1\}$	worker j is provisioned for tenant t over the horizon
$s_{fj} \in \{0,1\}$	snapshot of f pinned on j (tier L2)
$w_{fj} \in \mathbb{Z}_{\geq 0}$	warm sandboxes of f kept on j (tier L3)
$c_{tj} \geq 0$	slots of j earmarked for t (soft cap for the placer)
$S_j(H)$, p_j	survival probability of j's session over H; omission probability
e_j	effective availability $S_j(H)(1-p_j)$
a_j, σ_t, $a_{\min,t}$	assurance of j; tenant sensitivity; assurance floor
q_t, f_t	demand quantile (slots) to cover; number of defecting hosts to tolerate
cap_j, M_j	slots and memory of j
$d(j)$	failure domain of j (ASN and city)

E. Substrate profiles

VOSS separates the scheduler from the mechanism by which a host establishes trust. A *substrate profile* is data that tells one generic lifecycle engine how a worker establishes trust, how long it lasts, what the start-up tiers cost, what assurance the host carries, and how execution is slowed. We model five (Table V). The primary, `appliance_tpm`, is the TPM-attested appliance on consumer hardware, whose costs we measured (Section VIII-B). The server confidential-VM classes (`cvm_snp`, `coco_pool`) use ranges from the literature [26], [28], [33], [34], `drtm_hv` models a measured hypervisor launched under the owner's operating system, and `none` is the unattested reference used for baselines.

VI. TRUST-AWARE TWO-TIMESCALE SCHEDULING

This section is the core of VOSS. It first states the decision problem and its objectives precisely (Sections VI-A–VI-B), then says what information each decision uses and how it was obtained (Section VI-C), and then describes the slow planner, the fast placer and the estimators. Table VI lists the notation.

A. Decision problem and why two timescales

Setting. Within one cluster (tens of hosts, Section VI-G) the scheduler receives a stream of invocations and must (a) decide which hosts hold which attested session state, and (b) assign every invocation to a host. Its *inputs* are the cluster's state reported by workers, estimates learned from past outcomes, and configured policy (Section VI-C); its *outputs* are plan actions sent to workers (provision, pin, warm, release) and a dispatch message per invocation. The two kinds of decision differ in what they control and how fast they must be taken (Table VII).

A single per-invocation scheduler cannot manage attested session state, for three reasons. First, the security cost belongs to a group of hosts, not to one placement: a key released to a host stays for the whole session, so what matters is *how many different hosts* hold a tenant's key, and that count does not add up assignment by assignment. Second, robustness concerns the whole group: whether demand is still covered if any f hosts disappear cannot be read off from one placement. Third, establishing state takes seconds to tens of seconds (Fig. 1), far above a per-request budget, so it must be done ahead of demand. Placement, by contrast, is latency-critical and decomposes into an assignment over a bounded candidate set. Hence the slow loop decides, for the group of hosts as a whole, who holds which state, with an integer program or a heuristic, and the fast loop stays cheap.

What a plan contains, and when it runs. A plan says, for the coming horizon, which hosts hold which tenant's key (y), which function snapshots are pinned on which hosts' disks (s), how many *warm* sandboxes of each function are kept on each host (w: microVMs already started and initialised, idle in memory), and how many of a host's slots are earmarked for a tenant (c, a soft cap for the placer). Both loops run in the scheduler role: the placer for every invocation, the planner every $T_{\text{plan}} = 30\,\text{s}$ and when the situation changes materially. The value 30 s is a design choice, not a measured optimum. A plan's actions (a key release, a bundle fetch, a boot) take from a few hundred milliseconds to several seconds (Fig. 1), so the period must exceed that; sessions last minutes to hours, so it must be far shorter than that. We did not sweep it in the evaluation; a pilot at 15 s and 60 s (outside the reported experiments) showed no significant difference in P99.

B. What is minimised, and why not SLO violations directly

The quantity we ultimately care about. For a synchronous invocation i with deadline D_i, an *SLO violation* is the event $V_i = \{\text{the attempt fails}\} \cup \{\text{end-to-end latency} > D_i\}$. The deadline is $D_i = k_f\,(w_f(in_i) + 0.05\,\text{s})$, where w_f is the reference-core execution time of the function on that input and $k_f = 5$ (Section VIII-C). We *report* violations in every experiment (deadline-met fraction, success, P99). The scheduler's two loops do not minimise them in the same way (Table VII).

Fast loop: each invocation's expected violation. For invocation i of tenant t and candidate host j, let $b_i = D_i - (\text{now} - t_i^{\text{arrive}})$ be the remaining deadline budget and T_{ij} the predicted completion time (Section VI-E). The placer minimises

$$\begin{aligned} J_{ij} &= w_i\,P_{ij}^{\text{miss}} + \varepsilon\,\mathbb{E}[T_{ij}] + \lambda_{\text{exp}}\,x_{ij}, \\ x_{ij} &= \sigma_t(1-a_j)\mathbb{1}[j \notin \text{KH}(t)], \\ P_{ij}^{\text{miss}} &= p_{ij}^{\text{fail}} + \big(1-p_{ij}^{\text{fail}}\big)\,\Pr(T_{ij} > b_i), \\ p_{ij}^{\text{fail}} &= 1 - S_j(\bar{e}_{ij} + 2\,\text{s})(1-p_j). \end{aligned} \tag{3}$$

In words: P_{ij}^{miss} is the probability that invocation i violates its SLO if placed on j, with a failure (session end or omission)

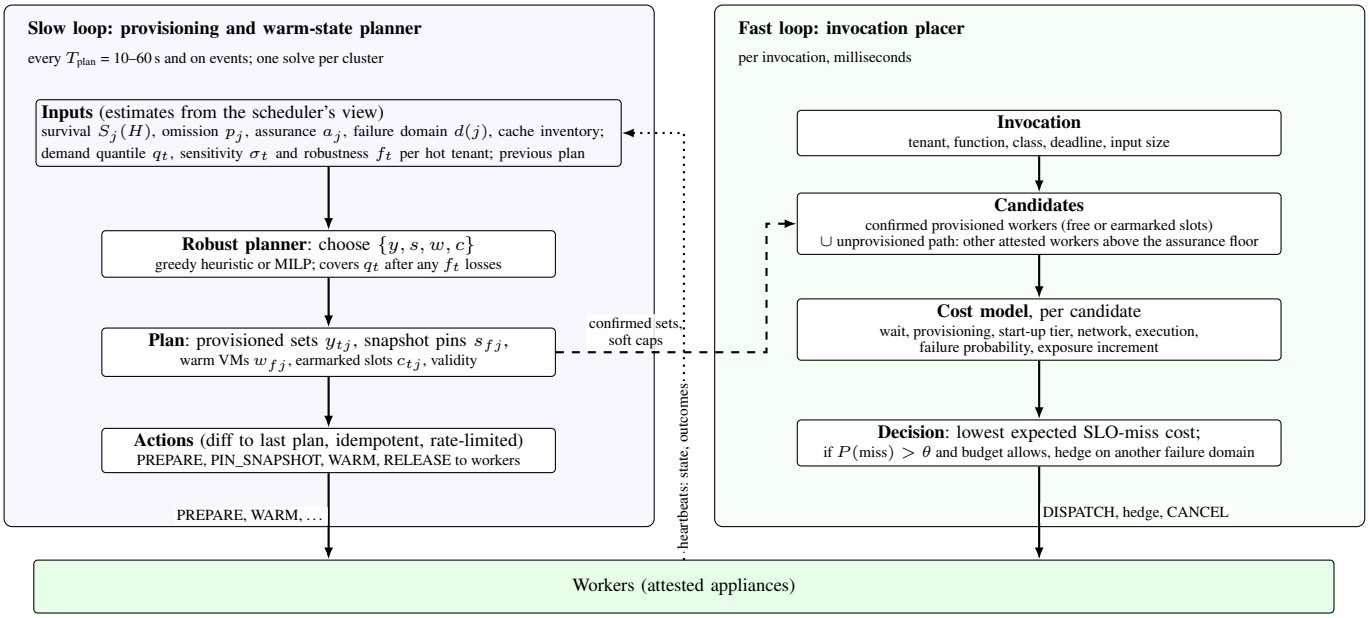


Fig. 6. The two scheduling timescales within a cluster. The planner decides where provisioned keys and warm state live, using estimates from the scheduler's view. The placer chooses the worker for each invocation from the planner's confirmed sets, using a per-candidate cost model. Workers confirm plan actions in heartbeats, so the placer never relies on a host that is not yet warm.

TABLE VII
WHAT EACH DECISION-MAKER MINIMISES, SUBJECT TO WHAT, AND WHETHER THAT IS SLO VIOLATIONS. DEFAULTS ARE IN TABLE VIII.

Decision-maker	Decides	Minimises	Subject to	SLO violations?
Fast loop (placer)	host for each invocation; hedge; admit or reject	$J_{ij} = w_i P_{ij}^{\text{miss}} + \varepsilon\,\mathbb{E}[T_{ij}] + \lambda_{\text{exp}} x_{ij}$ over hosts j (Eq. 3)	eligibility (valid attested session past the expected finish, not draining, assurance $\geq$ floor), top-K candidates, hedge budget	**Yes, per invocation:** P^{miss} is that invocation's violation probability; latency and exposure are regularizers
Slow loop (planner)	provisioned sets y, pinned snapshots s, warm VMs w, earmarked slots c	uncovered demand, donor memory and disk, key exposure, plan churn, less warm-start benefit (Eq. 4)	robust coverage of the demand quantile against any f_t defectors, slots, memory, disk, failure-domain caps, assurance floor, attestation freshness	**No, by design:** a surrogate of violation risk (invocations forced onto the unprovisioned path) plus costs SLOs do not see
Router	cluster for each request	expected completion time, plus a penalty on clusters whose observed miss rate exceeds θ, plus the price of an unprovisioned start (Section VI-G)	cluster not excluded; prefers free slots	Partly: observed miss rate enters as a penalty

counted as a violation; b_i is how much of the deadline is left; p_{ij}^{fail} is the chance that host j ends its session or omits before the run is over, where the survival probability S_j and the omission probability p_j are estimated from past outcomes (Section VI-F) and $\bar{e}_{ij}$ is the predicted mean execution time; x_{ij} is the exposure that placing i on j adds if j does not yet hold tenant t's key, where σ_t is the tenant's sensitivity, a_j the host's assurance score and $\text{KH}(t)$ the set of hosts that hold the key. The default weights are $w_i = 1$ for synchronous and 0.3 for asynchronous and filler work, $\varepsilon = 0.05\,\text{s}^{-1}$ and $\lambda_{\text{exp}} = 1$, so J is measured in *violation-equivalents*: one second of expected latency is weighted like a 5-point increase in violation probability, and giving a medium-sensitivity tenant's key to a new host of assurance 0.8 costs $1 \cdot (1 - 0.8) = 0.2$, the same as a 20% chance of violating the SLO. The placer therefore *does* minimise SLO violations for the invocation in front of it, and latency and key exposure are weighted against that objective, not the other way round. We chose per-invocation violation probability over a P99-based objective because P99 is a population statistic that cannot be attributed to one decision; minimising each invocation's own miss probability against its own remaining budget is the decision-level quantity whose mean is the deadline-miss rate and whose tail drives the P99. Expectation-, quantile-, CVaR- and tardiness-based alternatives are compared in an ablation (E03).

Slow loop: a surrogate, and why. The planner decides which hosts hold keys and warm state over the next horizon H ($H = 60\,\text{s}$; Table VIII). It does *not* minimise violations directly, for five reasons.

1) *Circularity.* Whether an invocation violates its SLO depends on the placer's later decisions, which depend on the plan; minimising violations would be a bilevel problem solved at every planning round.
2) *No per-request information.* At planning time the invocations do not exist. Only a forecast of demand exists, so the planner can reason about capacity relative to that forecast, not about individual deadlines.
3) *Rare, noisy events.* Violations are 1–5% events. An objective built from their empirical count has high variance; a coverage requirement at a fixed demand

TABLE VIII
DEFAULT PARAMETERS OF THE OBJECTIVES (ASSUMED UNLESS NOTED; SWEPT IN THE CITED EXPERIMENTS).

Parameter	Value
Planning period, horizon	$T_{\text{plan}} = 30\,\text{s}$, $H = 2T_{\text{plan}} = 60\,\text{s}$
Hot set (planned keys)	top 20 tenants, 50 functions, forecast rate ≥ 0.01/s
Demand quantile, robustness	$q_t = Q_{0.95}$; $f_t = \lceil 0.1\, N_{\text{cluster}} \rceil$
Planner prices	$\Lambda = 10$, $\rho_{\text{mem}} = 10^{-4}$/MB, $\rho_{\text{ssd}} = 2{\cdot}10^{-5}$/MB, $\lambda = 1$, $\kappa_{\text{switch}} = 0.05$, $\beta = 0.05$, $\gamma = 0.5$
Failure-domain cap	$\kappa_{\text{dom}} = 0.5$
Placer weights	$w_{\text{sync}} = 1$, $w_{\text{other}} = 0.3$, $\varepsilon = 0.05$, $\lambda_{\exp} = 1$, $\sigma \in \{0.1, 1, 3\}$
Candidates, hedge	$K = 12$; $\theta = 0.2$, budget 10%
Deadline	$D = 5\,(w_f + 0.05\,\text{s})$ for sync functions

quantile is stable and interpretable.

4) *Violations are not the only cost.* Warming everything everywhere would minimise violations and hold every tenant's key on every host. The scarce resources are donor memory and disk and key exposure (Section V-D), so the planner minimises cost *under a coverage requirement*, and the trade-off is reported as a Pareto front (Section IX-E).
5) *No hard guarantee exists.* Under churn and adversaries no plan can guarantee a deadline; coverage against *any* f_t defecting hosts is the strongest statement that assumes no model of the adversary.

How the planner still serves the SLO. The coverage constraint is a chance-constraint surrogate. If invocations of tenant t arrive as a Poisson process of rate λ_t with mean service time s_t, the number running concurrently is Poisson with mean $\lambda_t s_t$ (an M/G/∞ system), so $q_t = Q_{0.95}(\lambda_t s_t)$ slots suffice to start every invocation immediately with probability 0.95 under the forecast. Requiring provisioned, *available* capacity of at least q_t even after the f_t largest allocations are lost (Eq. 5) bounds the share of invocations forced onto the unprovisioned path, which is the dominant cause of violations on volunteer hosts. Uncovered demand (the slots of forecast demand that the provisioned hosts do not cover) is charged $\Lambda_t = 10$ per slot, two orders of magnitude above the price of memory (10^{-4} per MB, 0.05 for a 512 MB VM) and of exposure (about 0.2 per added key holder), so coverage is bought unless it is infeasible within the budgets, in which case the shortfall is reported. The Poisson assumption is the planner's *model*: simulated arrivals are a Markov-modulated process with $4\times$ bursts, so realized coverage is below 95% during bursts. This is a design rationale, not a theorem, and the realized violation rate is what the experiments measure.

What is deliberately not minimised. There is no explicit P99 target, no throughput objective, no fairness objective across donors and no provider cost term; cost against a commercial reference is reported afterwards (Section IX-G). Reputation never enters p^{fail} (Section VII-F).

C. Information model: what each decision knows, and how it knows

A reviewer should be able to tell, for every number a decision uses, whether it was observed, learned from history, forecast, or assumed. We classify every scheduler input into five kinds. **Observed (O)**: reported by an attested agent or trusted router by message, stale by at most the heartbeat period of 2 s plus network delay, with the scheduler's own dispatches since the last report added. **Learned from history (H)**: an estimator updated online from *past outcomes only*. **Forecast (F)**: a quantity computed from H-quantities for the future. **Calibrated (C)**: a constant or formula fixed before the run from the calibration bundle, the function catalogue or the geography model; *not* learned online. **Policy (P)**: configured by the tenant or operator. Table IX lists every quantity.

What is never used. The true session end time, the true run time and noise of an invocation, a host's true adversarial behaviour, future arrivals, and the true state of other hosts. The code base forbids the policy layer from importing the simulated world, which enforces this statically. A baseline that does use ground truth exists for analysis only and is never part of VOSS.

How much is history, how much is forecast, how much is assumed. The *slow loop is mostly forecast*: demand and survival over a 60 s horizon, extrapolated from rates smoothed over minutes and from session-length statistics accumulated over the whole run. The *fast loop is mostly observation*: occupancy, cache state and provisioning are at most a few seconds old, and only the execution-time distribution, survival over the next few seconds and the omission probability are statistical estimates learned from past outcomes. The *speculative* parts are exactly those marked C in Table IX: the scheduler assumes start-up costs and network times equal their calibrated nominal values, and that declared host speeds are exact. A deployment would measure these online; this prototype does not, which is a stated limitation (Section X-B). Sensitivity of results to prediction error or staleness (error-injection sweeps) is not part of the paper's experiments.

D. Slow loop: robust provisioning and warm-state planning

Inputs. Every $T_{\text{plan}} = 30\,\text{s}$ the scheduler builds a planning instance from its stale view (Table IX). The *pool* is the live hosts that are not draining, whose last heartbeat is under 7 s old and whose session has not expired, capped at 40 hosts (previous holders first, then slots $\times$ assurance). The *hot set* is the top 20 tenants and top 50 functions whose forecast rate is at least $0.01\,\text{s}^{-1}$; the long tail stays on the unprovisioned path (hosts without the key or warm state are used as they come), which keeps the program small. Keys are provisioned per tenant, because the secret belongs to the tenant, so the planner first decides which hosts hold each hot tenant's key, ranked by demand times cold penalty, and then, among the hosts that hold it, which functions to keep warm, ranked by start-up time saved per megabyte of memory. A tenant's hotness therefore decides where its functions may be warm, and a function's hotness decides whether it is kept warm. The evaluation workload has six tenants and 12 functions (Section VIII-A), so these limits do not bind there. For each hot tenant the planner receives demand q_t (the Poisson 95% quantile of forecast rate times service time); for each host the survival $S_j(H)$ over the horizon $H = 2T_{\text{plan}} = 60\,\text{s}$, the omission probability p_j, the assurance a_j, the failure domain, slots, memory, snapshot space and cache inventory; and tenant policy (σ_t, $a_{\min,t}$, f_t). All are *estimates*

TABLE IX
EVERY INPUT TO A SCHEDULING DECISION: ITS KIND (O OBSERVED, H LEARNED FROM HISTORY, F FORECAST, C CALIBRATED CONSTANT, P POLICY), ITS SOURCE AND UPDATE RULE, AND ITS VALUE BEFORE DATA EXIST.

Quantity	Used by	Kind	Source and update rule	Before enough data
Slots, memory and disk budgets, speed, link rates, location, tier, session start	planner, placer	O	declared by the worker agent at join; **speed and link rates are taken as exact**	–
Busy and queued slots, cache tier per function, provisioned tenants, expiry, drain deadline	planner, placer	O	heartbeat every 2 s; own dispatches since then counted as in flight; hosts silent for 7 s are left out of plans	host left out
Assurance a_j	planner, placer	O	issued by the verifier: substrate base $\times$ firmware-policy factor	–
Execution time on a host	placer	H	Bayesian log-linear regression per function, forgetting factor 0.995, per-worker offsets shrunk toward 0 (prior count 5), conformal rescaling of the spread over the last 500 residuals	catalogue prior (reference work, size exponent, $\sigma = 0.35$)
Wait for a slot	placer	O+H	queue length (busy + queued + in flight $-$ slots $+ 1)^+$ times the scheduler's EWMA of observed run times (weights $0.98/0.02$), divided by slots	1 s
Start-up and trust-establishment time per tier and substrate	placer, planner	C	means of the substrate profile (calibration bundle); **not re-learned online**	–
Network RTT and transfer time	placer, router	C	great-circle distance at 100 km per ms $\times 1.3$; bytes over the declared link rate; **no jitter, no measurement**	–
Session survival $S_j(\tau)$	planner, placer	H→F	per-tier Weibull by censored maximum likelihood on completed sessions and live ages, refit every 60 s; hour-of-day hazard multiplier after 48 session ends; drain notices only shorten	shape 0.7, scale 7200 s until 20 sessions of the tier ended
Omission probability p_j	planner, placer, detectors	H	Beta posterior of accepted invocations that return no result, mid-session only; CUSUM ($k = 0.1$, $h = 5$) resets a host to Beta$(2, 4)$	Beta$(0.5, 20)$, mean 0.024
Arrival rate per tenant and function	planner	H→F	counts in 10 s bins; EWMAs with $\tau = 60$ and 900 s; forecast is the maximum of both and of the regional rate pushed by routers	not planned until seen
Demand quantile q_t	planner	F	Poisson 95% quantile of rate $\times$ service time (M/G/∞); service time per key is an EWMA $(0.9/0.1)$ of observed run times	catalogue reference work
Warm-start benefit b_f	planner	F+C	rate $\times H \times$ (rootfs fetch + boot + init seconds saved) per warm unit	–
Tenant sensitivity, assurance floor, robustness f_t	all	P	tenant configuration	default $f_t = \lceil 0.1\, N \rceil$
Objective weights	planner, placer	P	configuration, assumed, swept in E04, E19, E23	–
Cluster load, observed miss rate, overhead, cold prices	router	O	scheduler summary every 1 s; cold prices are what the cluster *measured*	priors 3 s and 0.5 s

from the scheduler's view; no ground truth is used. **Output.** A plan $\{y, s, w, c\}$ with y_{tj} (host j holds the key of tenant t), s_{fj} (snapshot of f pinned on j), w_{fj} (warm sandboxes) and c_{tj} (slots earmarked for t on j), valid for H.

Objective. The planner minimizes uncovered demand, donor cost, exposure and plan churn, less the value of warm starts:

$$\begin{aligned}\min \sum_t \Lambda_t u_t + \sum_j \Big(\rho_{\text{mem}} \sum_f m_f w_{fj} + \rho_{\text{ssd}} \sum_f z_f s_{fj}\Big) \\ + \lambda \sum_t \sigma_t \sum_j (1 - a_j)\, y_{tj} \\ + \nu \sum_{t,j} \big(y_{tj} - y_{tj}^{\text{prev}}\big)^+ - \beta \sum_{f,j} b_f\, (w_{fj} + \gamma s_{fj}), \quad (4)\end{aligned}$$

where $u_t \geq 0$ is the uncovered demand of tenant t (the number of slots of its forecast demand q_t that the provisioned capacity does not cover), charged at Λ_t per slot, m_f and z_f are the memory and snapshot size of f, b_f the start-up time a warm sandbox saves, and the remaining coefficients $(\rho_{\text{mem}}, \rho_{\text{ssd}}, \lambda, \nu, \beta, \gamma)$ are weights that put memory, disk, exposure, plan churn and warm-start benefit in the same cost unit. Each term is logged separately so that latency, exposure and donor cost can be traded off explicitly (Section IX-D).

Robust coverage. Provisioned capacity must still cover demand when any f_t hosts defect together (crash, omit, or stall):

$$\begin{aligned}\sum_j e_j\, c_{tj} - \Big(f_t\, \theta_t + \sum_j \pi_{tj}\Big) + u_t \geq q_t, \\ \pi_{tj} \geq c_{tj} - \theta_t, \quad \pi_{tj}, \theta_t \geq 0 \quad \forall j.\end{aligned} \quad (5)$$

Here $e_j = S_j(H)(1 - p_j)$ is the effective availability of host j (the probability that its session lasts the horizon and that it does not omit), c_{tj} the slots of j earmarked for tenant t, q_t the demand quantile to cover, f_t the number of hosts to tolerate, and θ_t and π_{tj} auxiliary variables that encode the sum of the f_t largest allocations. The bracket equals the sum of the f_t largest allocations c_{tj}: for non-negative c, $\sum_{(f_t)} c = \min_{\theta \geq 0}\{f_t \theta + \sum_j (c_j - \theta)^+\}$ by LP duality, the standard reformulation of a cardinality-constrained uncertainty set [30]. Because $e_j \leq 1$, removing the f_t largest *slot allocations* removes at least as much as losing any f_t hosts removes from the coverage sum, so Eq. (5) is a sufficient condition and remains linear. The f_t hosts that are removed are chosen by worst case, so no probabilistic model of the adversary is needed for them: the constraint holds against f_t hosts with a perfect record, which is what an adversary who builds a clean history before defecting would be. The allocations that remain are weighted by the estimated availability e_j. By default $f_t = \lceil \alpha_{\text{assumed}} N_{\text{cluster}} \rceil$ and tenants may opt down.

Remaining constraints. Allocation requires provisioning ($c_{tj} \leq \text{cap}_j\, y_{tj}$, $w_{fj} \leq W_{\max}\, y_{t(f)j}$, $s_{fj} \leq y_{t(f)j}$). Slots, memory and snapshot storage of each host are respected. A *failure-domain cap* $\sum_{j \in D} c_{tj} \leq \kappa_{\text{dom}} \sum_j c_{tj}$ for every domain D prevents a correlated outage from removing most of a tenant's capacity. The assurance floor forces $y_{tj} = 0$ if $a_j < a_{\min,t}$, and a freshness constraint excludes a host whose attestation would expire within the horizon without time to re-quote.

Solving. One cluster has tens of workers and a bounded set of hot tenants, so the program has about 10^3–$3 \cdot 10^3$ integer

variables (a pool of at most 40 hosts, 20 tenants and 50 functions gives at most 800 key variables y, 800 allocations c, and 2,000 each of snapshot pins s and warm counts w). The greedy planner takes the same inputs and produces the same output. Its work is a few sorted passes over hosts and functions (8 ms per solve in our runs), whereas the exact program's solve time grows steeply with the pool: in a pilot outside the reported experiments the median solve took 0.3–1.1 s for 10 hosts, 4.7–5.0 s for 40 and 17–24 s for 80. We solve it with HiGHS [35] under a time limit, warm-started from the previous plan, falling back to the incumbent and then to a greedy heuristic. The *greedy planner* serves hot tenants in decreasing order of demand times cold penalty. For each tenant it repeatedly adds the host with the highest $S_j(1-p_j)a_j$ per unit of memory and exposure cost, subject to the assurance floor and domain cap, until coverage with the f_t largest contributions removed is at least q_t; it then adds warm sandboxes and snapshots in decreasing value density (start-up time saved per megabyte) until each host's memory or disk is used up. We adopt a decision rule fixed before the experiments: if the exact program improves P99 by less than 5% over the heuristic in all regimes, the heuristic is the method and the program is a reference (Section IX-C).

When the plan is recomputed. The plan is re-solved only if the instance changed materially: the hot set or the host pool differs, or $\ln(1+q_t)$ or the per-function warm-unit demand moved by more than a tolerance of 0.2 in any bucket. Otherwise the previous plan is reused for another horizon, and only unconfirmed actions are re-sent. Solve time and status are logged (`solver`, `plans`), and the modelled decision latency of a re-plan is charged to the clock.

From plan to actions. The plan $\{y, s, w, c\}$ is differenced against its predecessor into idempotent, rate-limited actions sent to workers: `PREPARE` (obtain the key from the KBS, fetch and decrypt bundles), `PIN_SNAPSHOT`, `WARM` and `RELEASE`. Workers confirm completion in heartbeats, and the plan is treated as valid only for state they have *confirmed*, so the placer never counts on a host that is not yet warm.

E. Fast loop: invocation placement

Inputs and output. For each invocation i the placer receives only metadata (function, tenant, deadline budget b_i, input size, idempotence, SLO class), the scheduler's cluster view (at most one heartbeat old plus its own dispatches) and the current plan. It outputs one of: a host j (optionally with a hedge host), *reject* or *defer*.

Candidates and feasibility. Candidates are the best $K = 12$ hosts by a cheap lower bound, drawn from (a) confirmed provisioned hosts and (b) other attested hosts (the *unprovisioned path*, which needs a key release and, usually, a start-up). A host is infeasible if it is draining or its session would expire within 5 s of the predicted completion, if its assurance is below the tenant floor, or if it lacks memory for a new sandbox.

Cost matrix. For each feasible pair (i, j) the model forms named components, logged per invocation: *wait* for a slot $\big(\text{busy} + \text{queued} + \text{in flight} - \text{slots} + 1\big)^+ \cdot \bar{s}/\text{slots}$ with $\bar{s}$ the EWMA service time; *trust* (zero if $K[t, j] = 1$, else key-release round trip plus policy check, calibrated); *start-up* (by the function's tier on j and its disk tier, calibrated); *network* (input and output over the geographic model); and *execution*, a log-normal from the Bayesian regression scaled by declared host speed and substrate slowdown. The completion time T_{ij} is a moment-matched shifted log-normal over these components. The failure probability is $p_{ij}^{\text{fail}} = 1 - S_j(\bar{e} + 2\,\text{s})(1 - p_j)$, with $\bar{e}$ the predicted mean execution time: the host must survive the run and not omit. Failure counts as a violation in Eq. (3), so the cost trades a host that is fast but about to leave against one that is slower but safe.

Placers. The default placer assigns each synchronous invocation to the feasible host with the lowest J_{ij} (Eq. (3)). For asynchronous and filler work, which arrives in small windows, a minimum-cost flow (`linear_sum_assignment`) maps window invocations to (host, slot) pairs in polynomial time; the window is 0.2 s. A synchronous invocation is placed immediately, and invocations already routed to a host and still in flight count against its free slots (the wait term above), so a host is judged by its current load and not only by how good it is. All baseline placers use the same candidates and cost matrix, so differences are due to the rule alone.

Admission, hedging and retries. *Admission*: if every candidate has $P_{\text{miss}} \geq 0.95$ *and* even the shortest slot wait exceeds the remaining budget (overload), a synchronous request is rejected and an asynchronous one deferred; misses caused only by cold start-up are admitted, because they warm the caches later requests need. *Hedging*: if the best candidate has $P_{\text{miss}} > \theta = 0.2$, the function is idempotent and the cluster's hedge budget (10% of placements, token bucket) allows, a tied duplicate goes to the best candidate in a *different failure domain*; the loser is cancelled on the first result [29]. Attested receipts make duplicates integrity-safe, but not effect-safe: hedging and retries apply only to *idempotent* functions (calling the function twice has the same effect as calling it once; each function carries this flag), and for other functions a lost result is reported as a failure. *Retries*: on a watchdog or a failure the invocation is re-placed with the failed host excluded, preferring provisioned hosts, within a per-invocation budget and a per-cluster retry token bucket, and only if the function is idempotent or the failure occurred before execution began.

F. Estimators

Four estimators feed both loops and are trained online, using only outcomes visible to the scheduler. Parameters are in Table IX. *Survival* $S_j(\tau)$ is a Weibull distribution per tier fitted by censored maximum likelihood on completed sessions and live session ages, refit every 60 s from a prior (shape 0.7, scale 7200 s) until 20 sessions of the tier have ended, with an hour-of-day hazard multiplier once 48 have ended; drain notices only shorten predicted life, since they may be fake. *Omission* p_j is a Beta posterior over accepted invocations that return a result, counting mid-session outcomes only so that session ends are not double counted; a Beta-CUSUM change-point detector ($k = 0.1$, $h = 5$) resets a flagged host's eligibility and its posterior to $\text{Beta}(2, 4)$, which defeats hosts that behave until they defect. *Execution time* is a Bayesian log-linear regression per function with forgetting factor 0.995 and shrinkage of per-worker offsets, and a conformal rescaling of the spread over

the last 500 residuals [36]. *Arrivals* are forecast per hot tenant and function from counts in 10 s bins by EWMAs with time constants 60 and 900 s; the forecast is the maximum of both and of the regional rate reported by routers. The estimators keep a few numbers each, not a history table: one regression per function with one offset per worker, two Weibull parameters per tier, a Beta pair per host and the arrival counts. They live in the scheduler's memory and are checkpointed to the control plane every 30 s (Section VII-C); there is no stored record of past runtimes for every (function, worker) pair. What these estimators do *not* learn is stated in Section VI-C: start-up and trust costs, network times and declared speeds are calibrated constants. The planner's robustness (Eq. (5)) does not depend on the omission estimator being right; the estimator only reduces how often robustness is exercised.

G. Cluster arrangement and routing

Clusters keep the planner small and intra-cluster latency low. The orchestrator groups workers into geographic clusters by k-medoids over a round-trip-time embedding with $k = \lceil N/N_{\text{target}} \rceil$ (target size 30, between 10 and 60 hosts), requires each to contain enough failure domains and assurance mass for robust plans to be feasible, and adjusts them on triggers (size bounds, SLO health, demand imbalance, forecast, diversity deficit, scheduler loss) with hysteresis, dwell times and a migration cost, since a moved worker loses its provisioned state. A router (one per region, trusted for availability) scores each cluster c for an invocation of tenant t and function f by

$$\begin{aligned}\text{score}(c) = {} & \text{rtt}_c + o_c + \text{backlog}_c + h_s \max(0, \hat{m}_c - \theta) \\ & + \pi_c(t)\,[\text{no provisioned capacity}] + \text{cold}_c(f),\end{aligned} \tag{6}$$

where every term is in seconds of expected delay. rtt_c is the round-trip time from the request's origin to cluster c, o_c the cluster's measured scheduling overhead, backlog_c the queueing delay when the cluster is full, and the health term charges h_s seconds per unit of observed miss rate $\hat{m}_c$ above θ (the cluster sends its summary every 1 s). $\pi_c(t)$ is charged only when c has no provisioned capacity for tenant t: the key-release time that c has measured plus an exposure charge. $\text{cold}_c(f)$ is charged only when c has no warm sandbox of f: the cold-start time c has measured for it, taken from its recent starts (priors of 3 s and 0.5 s until data exist). The score is evaluated per request on the latest summaries, not over a long window. Spillover to another cluster is therefore a decision with an explicit cost in seconds, and a router falls back to the next-best cluster when the dispatch watchdog fires (1 s).

VII. Fault Tolerance and Accountability

By Property 1, one set of mechanisms must handle crashes, churn and malice. This section describes how failures are detected, how schedulers are replaced without consensus among volunteers, and how work is accounted.

A. Fault model and the progress principle

Table X lists the failures the mechanisms must handle. The hard cases are those where *liveness signals look healthy* (a liveness signal is a periodic heartbeat that says the host is still alive): a *black-holing* host (the networking term for a node that silently swallows what it receives) keeps sending heartbeats while dropping the work it accepted, an on–off host builds a clean record before defecting, and a drain notice may be a lie. We therefore adopt the principle that *liveness signals from untrusted hosts are hints, and progress observed by trusted or attested parties is evidence*. The trusted observers are the routers; the attested observers are the worker agents and the scheduler code, whose *hosts* are still untrusted for availability.

TABLE X
Failures handled and what makes each hard to detect.

Failure	Source	Difficulty
Crash, session end	owner, power	easy: channel and heartbeats stop
Graceful drain	owner	announced, but the announcement may be fake
Black hole, selective omission	malicious owner	**heartbeats look healthy** (gray failure)
On–off defection	malicious owner	history is misleading
Delay, throttle	malicious or overloaded	looks like slowness
Partition	network	a stale scheduler keeps acting
Replay	malicious owner	rejected by TLS sequence numbers

TABLE XI
Progress-based detectors. Heartbeats are a fast path only.

Detector	Observer	Signal	Suspects
Dispatch watchdog	router	ENQUEUE without ACCEPTED in T_{dispatch}	scheduler
Result watchdog	scheduler	ACCEPTED without RESULT by q90 + margin	worker
Cluster stall	router → orchestrator	repeated dispatch timeouts	scheduler
Decision latency	orchestrator	attested placement time $> T_{\text{decide}}$	slow scheduler
Outcome rate	scheduler	omission rate, CUSUM change point	worker
Heartbeat	scheduler	missing heartbeats	crash only

B. Progress-based failure detection

Table XI lists the detectors. Each watches an event that must occur if the component is working and attributes a missing event to a suspect. The router's *dispatch watchdog* suspects a scheduler (or its cluster) when an enqueued invocation is not acknowledged within T_{dispatch}. The scheduler's *result watchdog* suspects a worker that has accepted work but not returned a result by the predicted completion quantile. A *cluster-stall* report from routers and a *decision-latency SLO* on the scheduler's attested timestamps catch a scheduler that is alive but slow. The *outcome-rate* model (Section VI-F) catches selective and on–off behavior. Heartbeat and accrual detectors [20] are kept only as a fast path for crashes and are never sufficient alone. A suspected worker receives no new work, its in-flight work is hedged or retried, and it loses provisioned eligibility; a suspected scheduler triggers failover.

C. Scheduler failover without volunteer consensus

Replicated state machines among volunteers would be slow and would reintroduce an agreement problem on hosts that may be adversarial. VOSS avoids it because the provider-operated

TABLE XII
SOFT SCHEDULER STATE AND ITS RECOVERY PATH.

State	Held by	Recovery
Undispatched invocations	router pending table	router resends
Running and queued work	workers	roll call (query workers)
Plan (provisioned sets, pins)	workers, checkpoint	roll call, recompute
Predictor, survival, omission posteriors	checkpoint	checkpoint, replay
Work records	router receipts	nothing to recover

orchestrator is a reliable arbiter, and a scheduler's state is *soft*. Fig. 7 shows the sequence.

Appointment and fencing. The orchestrator appoints a scheduler by an atomic compare-and-swap on the cluster's *epoch* in its database, in the manner of a lease [37]. The appointment token binds the cluster, the epoch and the *session-certificate identifier* of the appointee, so a cloned appliance with the same image but a different session key is not appointed. Workers accept dispatches only from the current (epoch, certificate) pair and reject lower epochs with `STALE`, which acts as a fencing token. A scheduler that cannot reach the orchestrator for a lease period stops dispatching on its own, and workers fence it anyway. A partitioned scheduler can therefore cause only duplicate work, which is within at-least-once execution. That is safe for idempotent functions; for a non-idempotent function a duplicate dispatched within the lease period could repeat its effect, which the protocol does not prevent (Section X-B).

Soft state. A scheduler keeps no durable state on its host (Table XII). Undispatched invocations are held by routers, running work and confirmed provisioning by workers, learned estimator state in a control-plane checkpoint refreshed every few tens of seconds, and work records in receipts. After appointment, the successor installs the epoch and performs a *roll call*: it asks workers for their running and queued work, confirmed pins and keys, retained outcomes and cache inventory, rebuilds its view, replays outcomes, and re-plans from the checkpoint. Because nothing authoritative lives on the old host, a rolled-back or restarted appliance is a new instance that must be re-appointed, and its stale memory is never trusted.

Succession list. The orchestrator keeps a ranked list of candidates by survival, speed, centrality and assurance, with failure-domain diversity and hysteresis. Every attested worker is a candidate, because the scheduler role is in the image, so promotion costs no boot (a better-ranked, more reliable host is preferred, but any worker can serve); a warm standby also receives heartbeats. Planned handovers on a drain notice or decision-latency degradation flush a checkpoint before the swap; a *fake* drain by a malicious scheduler host just causes a clean handover.

D. Worker failures

Session end discards keys and warm sandboxes; the invocation is retried or hedged elsewhere, preferring provisioned hosts. A per-cluster retry token bucket and router spillover contain retry storms during regional waves. Drain and preemption notices from the owner side only make the scheduler more conservative (stop new dispatch, shorten predicted life) and never excuse an omission in the estimators. A change point or a watchdog streak marks a host suspected, revokes its eligibility and triggers re-planning.

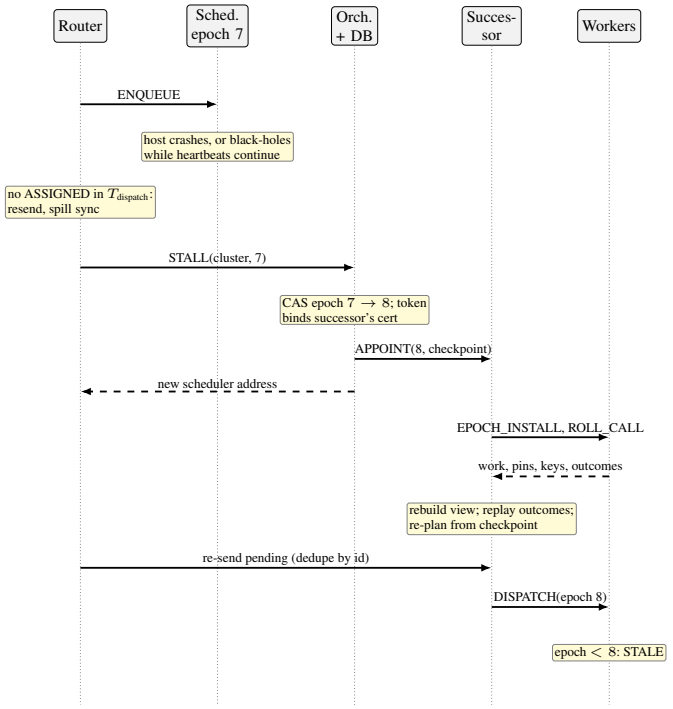


Fig. 7. Scheduler failover. The router's dispatch watchdog detects a crashed or black-holing scheduler; the orchestrator swaps the epoch atomically and appoints a successor, which reconstructs soft state from workers and a checkpoint.

TABLE XIII
FAULT-TOLERANCE PARAMETERS (DEFAULTS).

Mechanism	Default
Dispatch watchdog T_{dispatch}; sync spill T_{spill}	1 s; 1.5 s
Result watchdog	predicted q90 $\times 1.2$, at least 3 s, then a probe with 2 s timeout
Cluster stall report	3 dispatch-watchdog fires within 5 s
Decision-latency SLO $T_{\text{decide,max}}$	200 ms
Suspicion (watchdog streak; duration)	2 misses; 600 s
Change point (Beta-CUSUM)	$k = 0.1$, $h = 5$
Hedge threshold θ; budget	0.2; 10% of placements
Scheduler lease; checkpoint period	5 s; 30 s
Warm standby schedulers	1

E. Inputs, outputs and parameters

Fault handling is driven only by observable events. Its *inputs* are messages with timestamps (enqueue, accept, result, probe replies, heartbeats, outcome records) and the estimators' predicted completion quantiles; its *outputs* are suspicion flags, re-placements, hedges, scheduler appointments and revocations of provisioned eligibility. Table XIII gives the defaults; all are configuration, tagged assumed or literature, not measured.

F. Attested receipts and credits

Each result carries a receipt signed with the session key: invocation and attempt identifiers, worker and certificate identifiers, epoch, execution time, *reference work units*, and a digest of the output. The router verifies it against the certificate (and the verifier's revocation list) and appends it to the operational database. A host cannot claim work it did not do; it can only decline to do work, which is an omission handled above. Credits are proportional to reference work units, not elapsed time, so throttling earns nothing, and a small credit rewards cache pinned by the plan while the host answered probes. Reputation is kept apart from the survival and omission estimators, which have disjoint inputs (session ends versus mid-session omissions), so that no signal is counted twice; it aggregates declared-commitment behavior over the long term and is used for rewards and as a placement tie-breaker only. We deliberately omit a blockchain: receipts are already verifiable by third parties, and the only remaining risk is the control plane withholding records.

VIII. Simulator, Calibration and Evaluation Protocol

We evaluate VOSS in a deterministic discrete-event simulator, because a volunteer platform with thousands of hosts, controlled churn and adversaries, and paired comparison of policies cannot be run on real machines. This section describes what the simulator models, how its parameters are calibrated, and the protocol used for every comparison.

A. Simulation methodology

Technique and technology. The simulator is a custom single-threaded *discrete-event simulator* written in Python 3.11. Simulated time advances only by popping the earliest entry of a binary-heap event queue ordered by $(\text{time}, \text{sequence number})$; each entry is a callback, and cancellation is lazy. We did not use an existing framework (e.g. SimPy) because the kernel is under 70 lines and a single total order of events is what makes runs exactly repeatable. Numerical work uses NumPy (random streams are PCG64 generators seeded from a SHA-256 of the seed, a stream name and keys), SciPy (HiGHS for the MILP, `linear_sum_assignment` for the flow placer and special functions for the estimators), pandas and PyArrow (Parquet logs), Pydantic (frozen configuration tree) and PyYAML. A run is one process; independent runs (seeds, grid cells) are distributed over CI jobs.

Inputs, run, outputs (Fig. 8). The input of a run is a YAML configuration (scenario, systems, policy names, seed) and a calibration bundle. *Scenario generation* draws, from named seeded streams, all *exogenous* traces before the clock starts: the worker table, every session start and end, adversary assignments, the arrival process and the per-invocation execution noise. They are independent of the system under test and content-hashed, so two systems compared on one seed face identical worlds. The *engine* then instantiates the actors (workers, scheduler roles, routers, orchestrator, verifier, tenant key servers) and the policies, and processes events until the horizon. Policies see only immutable views assembled from *messages*, never the world objects. At the end, *invariant checks* assert that every success was executed on an attested host after key release, that each invocation has one terminal outcome, that no stale-epoch scheduler dispatched, that time components sum to the end-to-end latency, and that the receipts are complete. The *Recorder* writes raw Parquet tables (per-invocation records with time components, events, plans, solver calls, cache transitions, samples, messages, receipts) and a manifest with the scenario, calibration and result hashes. Offline analysis reads only these tables.

TABLE XIV
What the simulator computes, takes from calibration, or assumes.

Class	Content
Simulated (emergent)	queueing, contention on access links and disks, cache state and evictions, session boots, attestation and key release, message delays, failures, detection and failover, hedging waste
Calibrated (measured, Sec. VIII-B)	start-up and snapshot times, TPM quote and boot times, disk speeds, function reference work (SeBS profiles)
Literature (swept)	confidential-VM and DRTM costs, Firecracker figures
Assumed (swept)	session-length and churn parameters, adversary fractions and strategies, arrival intensity, assurance bases, objective weights

Event loop and what costs time. Every interaction between nodes is a typed message sent through the network model, which delivers it after a delay from the great-circle distance between about 100 metropolitan areas (100 km per ms, route factor 1.3) plus last-mile terms and lognormal jitter (the scheduler's own RTT estimate has no jitter, Table IX). Bulk transfers share each host's access link by processor sharing. Nothing is free (Table XIV): disk-tier reads, CPU contention, TPM quotes, verification, key release and the decision latency of planner, placer, router and orchestrator all consume simulated time. Decision latency is modelled as $a + b \cdot \text{work}$ per decision kind (for the planner, work is the number of program variables; for the placer, candidates times components), so results do not depend on the speed of the simulating machine. The coefficients are literature-seeded placeholders, not measurements (tagged for refit from the scalability experiment). For the planner the charge is $0.02\,\text{s} + 50\,\mu\text{s}$ per program variable, about 0.1 s for the programs here, for the greedy and the exact planner alike; the exact planner's measured solve time (4.1 s) is reported but not charged.

World model. Workers are drawn from three tiers (nightly, idle-hours, dedicated) with heterogeneous hardware, home cities and substrates. Sessions follow log-normal, Weibull or nightly-schedule length distributions with a time-of-day hazard; a fraction of departures is graceful and the rest abrupt, and correlated shocks remove a share of a city at once. Three churn regimes (low, medium, high) plus burst and correlated variants parameterize these. No public trace of consumer donation sessions exists, so the shapes follow volunteer-availability studies [19] and the parameters are tagged *assumed* and swept. The workload has six tenants and 12 functions modeled on SeBS [38] in sync, async and filler classes, Zipf-skewed popularity, a Markov-modulated Poisson arrival process with bursts and diurnal modulation [1], and flash crowds as a variant. A synchronous invocation's deadline is $k = 5$ times its warm

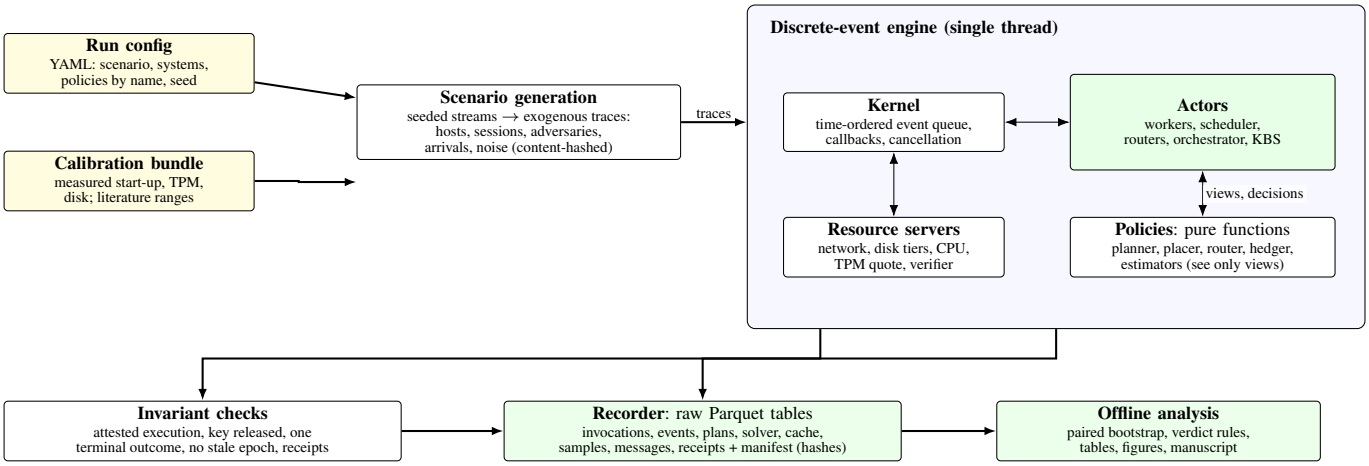


Fig. 8. Simulation pipeline. A run is a pure function of configuration, calibration bundle and seed: exogenous traces are generated first, then a deterministic event loop executes actors and policies, and only logs leave the engine.

TABLE XV
CALIBRATION ENTRIES (S UNLESS NOTED): PLACEHOLDER VERSUS MEASURED ON THE CALIBRATION LAPTOP. RATIO IS MEASURED OVER PLACEHOLDER.

Entry	Placeholder	Tag	Measured	Ratio
TPM quote	0.25	literature	0.5062	2.0×
Verifier (check quote + signature)	0.01	assumed	0.03985	4.0×
SHA-256 per MB	0.0007	literature	0.003714	5.3×
Firecracker boot to init (warm cache, HDD)	0.15	literature	0.1542	1.0×
Firecracker snapshot load (warm, HDD)	0.02	literature	0.006767	0.3×
Snapshot create (512 MB, HDD)	0.2	literature	6.68	33.4×
Reboot to kernel start (firmware+loader+kernel)	15	assumed	21.05	1.4×
Kernel start to agent ready	4	assumed	30.74	7.7×

latency. Adversaries are a fraction α of workers that behave as omitting (black-hole, random omission, on–off, throttling, fake drain notices, or a colluding failure domain); by Property 1 none forges results or reads keys on an attested substrate, while the `none` substrate admits both.

Reproducibility. A run is a function of its scenario and seed, and its raw tables carry a result hash recorded in a manifest. Section VIII-D states how results are reproduced, on which hardware, and what was verified.

B. Calibration

Parameters in the specification are literature values or assumptions, tagged as such. Measured values come only from calibration bundles. We measured one on a consumer laptop with an Intel Core i3-7020U CPU, a discrete TPM 2.0 and a hard disk, which stands in for a donor node. A harness reboots the machine into the appliance and records wall-clock timestamps (resolution about 1 s), and microbenchmarks cover TPM quotes, hashing, Firecracker boot and snapshots. Table XV compares measured values with the pre-measurement placeholders. Reboot to kernel is slower (21 s against 15 s), the kernel-to-agent phase far slower (31 s), and snapshot creation on a hard disk costs 6.7 s for 512 MB. From power-on, an attested worker takes 65 s (median of 9 boots; 44 s of it after kernel start). A warm-cache microVM boot (154 ms) is close to the literature [2].

Two limits apply. The machine is not the intended donor node and lacks AMD SEV-SNP and Intel TDX, so confidential-VM and DRTM parameters are literature values [28], [33], [34], which we sweep. And the appliance was exercised in *host mode* (the TPM measurement chain and quote path on the host, with microVMs through Firecracker) rather than as a locked image on bare metal, so the attested-session figure is a measured sum, not a measured attestation of the final image. Simulator output has not been validated end to end against a multi-machine testbed (Section X-B).

Validation against one machine (E01). To check the start-up and execution parts of the simulator against real execution we ran the same scenarios in a single-box emulation on a lab node (Intel Core i7-12700, 125 GB, NVMe, Firecracker 1.10.1) and in the simulator, with a calibration bundle measured on that node (the session boot keeps its placeholder on both sides). The emulation has 16 donor slices of two slots each, five functions, ten minutes per run and seeds 1–3. It replaces boot, snapshot restore and creation, dispatch and function execution by real microVMs; the network, key release, attestation, queueing and the TPM stay models on a wall-clock kernel, and the policy code runs for real with its measured decision time. Success rate agrees within 0.6 percentage points in every scenario and the mix of start-up levels is identical (total variation 0.00); median sync latency agrees within 6–23%. The shape of the latency distribution does not meet our pre-set cut-offs (Kolmogorov–Smirnov distance 0.20–0.51), so every scenario is graded "partly" close and none "close". At light load the simulator's P99 is about 3.7 to 4.1 times the emulation's on seed means, driven by a handful of cold invocations (the P99 of about 530 synchronous requests is the five slowest); at 100 inv/s the P99 differs by -3 to -6%. E01 does not test the network, attestation, key release, the session boot, detection and failover at scale, or the hard-disk tier.

C. Protocol

Each experiment is a frozen specification (base scenario, systems, grid) and *every* cell is run with seeds 1–10 (three seeds for the two largest scalability points, which take up to 20 wall-clock minutes and 9 GB). Unless stated, VOSS uses the greedy planner and the default placer, router and

hedger; the exact MILP is used only in the planner comparison (E04) and the Pareto analysis (E23). Systems in a comparison share the scenario and the per-invocation random streams, the *common random numbers* technique of simulation analysis, so each comparison is *paired* by seed: for metric m, we form $\Delta_s = m_A(s) - m_B(s)$ per seed and report the mean with a 95% percentile bootstrap interval [39]. Percentiles are computed per run first, then averaged across seeds, never pooled. Relative changes use the baseline mean as the denominator.

Each hypothesis is stated before the run together with a mechanical verdict rule: *supported* if the interval excludes zero in the predicted direction and the mean effect is at least a stated margin; *weak* if it excludes zero but the effect is below the margin; *inconclusive* if it includes zero; *refuted* if it excludes zero in the opposite direction. The manuscript's numbers and tables are generated from the result tables by a script that also fails if a result stops supporting the sentence it backs.

Metrics. We report success (completed invocations), the deadline-met fraction, P50 and P99 sync latency, *waste* (extra accepted attempts per success: retries, hedges and adopted re-dispatches), donor memory (GB held warm), and key exposure (Eq. (2), comparable only within one substrate mix). **SLO feasibility.** A system is *feasible* in a scenario cell if the lower 95% bound of its success is at least 0.99 and either the lower bound of its deadline-met fraction is at least a target or its P99 is below an absolute target of 2, 3 or 5 s. The feasibility maps of Section IX-B thus state *where* each design works rather than an average.

Baselines. Baselines use the same simulator, candidates and cost model and differ only in the registered policy. For planning: *no planning* (reactive: hosts are provisioned only when an invocation arrives), *keep-alive TTL*, *greedy planner*, and MILP ablations. For detection: heartbeat-only. For robustness: $f = 0$. The *simple baseline* combines static regional clusters, nearest-region routing, least-loaded placement, keep-alive and no hedging; a variant runs without attestation (substrate `none`), which is what volunteer FaaS does without security. Commercial-style references (*Lambda-like*, *Azure-like*) are always-on datacenter hosts in the router metros with per-function keep-alive (10 and 20 min) and nearest-region routing, priced from list prices; they run on the same engine so differences stem from policy and substrate, not the simulator.

D. Reproducibility and requirements

Every number, table and figure in Section IX is regenerated from raw run logs by committed scripts, and the runs themselves are *bit-reproducible*: re-executing a run gives raw tables with the same result hash. Floating-point simulation is bit-reproducible only under stated conditions, so Table XVI lists them and Table XVII reports what we checked.

Mechanisms. All randomness comes from named, seeded streams, events at equal times are ordered by a fixed key, and iteration order is sorted wherever it can affect results. Each run writes a manifest with its scenario hash, the calibration-bundle hash, a numeric canary (below) and a SHA-256 result hash over the raw tables; `voss repro` re-runs a manifest and asserts the hash, and the hashes of all runs in this paper are committed with the artifact. Analysis reads logs only.

TABLE XVI
REQUIREMENTS TO REPRODUCE THE RESULTS BIT-FOR-BIT.

CPU	x86-64 with fused multiply-add (every Intel and AMD CPU since 2013). AVX-512 is neither required nor used: the pin selects the SSE4 baseline, so AVX2 and AVX-512 machines agree.
Operating system	Linux with the GNU C library (tested: Ubuntu 22.04 and 24.04, glibc 2.35 and 2.39).
Software	Python 3.11 and the dependency versions in the committed lockfile (NumPy 2.x, SciPy with HiGHS, pandas, PyArrow); `NPY_DISABLE_CPU_FEATURES="X86_V4 X86_V3 AVX512_ICL AVX512_SPR"`, which the package, the Makefile and the CI workflows set.
Special hardware	None: no TPM, reboot or BIOS access. The calibration bundle is committed; the reboot-based calibration harness (Section VIII-B) is not part of reproduction.
Compute	21 experiments, 2,366 runs. About four hours of wall-clock time as 21 parallel jobs on GitHub-hosted runners; compressed raw logs are 0.1–0.7 GB per experiment.
Not covered	ARM, other operating systems and other C libraries are not bit-comparable; for them the portable claim is statistical (ten paired seeds). Wall-clock columns (solver time, simulated seconds per second) vary by machine, are informational and are excluded from the hashes.

TABLE XVII
REPRODUCTION CHECKS. A RUN *matches* IF ITS RESULT HASH EQUALS THE RECORDED ONE.

Check	Machines	Outcome
Before the pin (one experiment, 50 runs)	GitHub runners with AVX-512 versus AVX2-only CPUs	0 of 50 match
After the pin (probe experiment)	Intel Xeon Platinum 8573C; AMD EPYC 9V74, 9V45 and 7763 (the last without AVX-512); Ubuntu 22.04 and 24.04	identical hashes on all
Full sweep, re-executed	independently provisioned GitHub runners, all 21 experiments	2,366 of 2,366 match; all tables identical

Why the CPU matters. A last-bit difference in a prediction can flip a near-tie in placement, after which the whole trajectory diverges, so the arithmetic must not depend on the processor. We found two sources and removed both. BLAS kernels differ between microarchitectures, so the small regression in the execution-time estimator uses fixed-order exactly rounded operations (`math.fsum`) instead. NumPy's transcendental functions (`exp`, `log`, `power`, and others) have separate AVX-512, AVX2 and SSE4 implementations whose last bits differ, and GitHub assigns both kinds of CPU to jobs of one workflow. A first verification rerun failed for this reason (no run of one experiment matched, and table cells differed by up to 7.7%). We now pin NumPy's SIMD dispatch to the SSE4 baseline, which every x86-64 CPU supports, before NumPy is imported, and each manifest records a canary hash of the transcendental functions on fixed inputs; `voss run` warns if it differs from the reference.

Reproducing. With the lockfile installed, the following regenerates the runs, compares them with the recorded hashes, and rebuilds this document.

```
uv sync --frozen
make paper          # 21 specs x 10 seeds
make paper-verify   # compare hashes
voss repro runs/E09/<run>/manifest.json
make manuscript
```

IX. EVALUATION

We organize the results around the questions the design raises. Table XXVIII at the end lists every hypothesis with its

TABLE XVIII
SIMULATED SESSION ESTABLISHMENT AND START-UP LATENCY (S) FROM EACH TIER (L0 NOTHING CACHED, L3 WARM VM; "–" NOT APPLICABLE). PRE-MEASUREMENT PARAMETERS (TABLE XV).

Substrate	Session	L0	L1	L2	L3
appliance (TPM)	19.2	1.45	0.346	0.0285	0.00197
DRTM hypervisor	2.31	1.44	0.34	0.0284	0.00196
CVM (SEV-SNP)	1.41	2.31	1.19	0.155	0.002
CoCo pool	8.11	2.25	1.14	–	0.002
no attestation	1.06	1.45	0.344	0.0278	0.00201

TABLE XIX
NUMBER OF CELLS IN WHICH VOSS IS FEASIBLE, BY TARGET (DEADLINE-MET FRACTION, OR ABSOLUTE SYNC P99).

Panel	0.90	0.95	0.98	2 s	3 s	5 s	Cells
	deadline-met target			sync P99 target			
A: churn × α	9	9	0	8	9	9	9
B: substrate × load	16	11	0	6	13	16	16
C: N × load	8	8	0	8	10	10	12

verdict.

- RQ1 What does establishing trust cost on each substrate, and how does that shape start-up?
- RQ2 Where is volunteer FaaS feasible, and where is it not?
- RQ3 Does planning attested session state pay for itself?
- RQ4 Does progress-based detection make malicious owners look like churn?
- RQ5 What do robustness and exposure cost?
- RQ6 Which designed components do not earn their complexity?
- RQ7 How does VOSS compare with simple baselines and a commercial reference, and how does it scale?

A. RQ1: Cost of establishing trust across substrates

Table XVIII gives, for each substrate, the simulated time to establish a session and the start-up latency from each tier. Two observations shape everything that follows. First, the appliance pays a large one-off cost per session (19 s with the pre-measurement parameters; 65 s from power-on to an accepted quote on the calibration machine), compared with 1.4 s for a confidential VM and 8.1 s for a confidential-container pool, and loses all session state when the session ends. Second, once the session exists the appliance is as fast as an unattested Firecracker host: warm and snapshot starts are within a few percent of the unattested figures (H5, supported), because attestation is paid per session, not per invocation. Confidential VMs invert the trade, with cheap sessions but slower lower tiers. Our expectation that confidential-VM classes establish trust 10–100× *slower* than the appliance was refuted: they are about 93% faster, since the appliance's cost is a reboot.

B. RQ2: Where is volunteer FaaS feasible?

Averages hide the regimes in which a design fails, so we map feasibility over three two-dimensional panels. Fig. 9 colors each cell by which of VOSS and the best simple baseline is feasible under the default criterion (success ≥ 0.99 and deadline-met ≥ 0.95) and prints VOSS's P99. Table XIX counts VOSS-feasible cells under other targets.

VOSS is the *only* feasible system in all 9 cells of panel A, which spans three churn regimes and up to 20% omitting owners, in 9 of 16 cells of panel B and in 4 of 12 cells of panel C. With an absolute sync P99 target of 3 s it is feasible in 9 of 9, 13 of 16 and 10 of 12 cells, and the simple baseline in none; at 5 s VOSS is feasible in 9, 16 and 10 cells. The two criteria measure different things (a deadline is a multiple of warm latency, the absolute target is what a user perceives), which is why a baseline can pass one and fail the other.

The map also shows where VOSS is *not* the right choice. On confidential-VM and confidential-container substrates at higher load the simple baseline is the only feasible system in 4 cells (Fig. 9b). We conjecture that with cheap session establishment there is little cold-start cost left for planning to remove while it still adds exposure and memory; we did not test this. With 100 workers no system is feasible in any cell of panel C (4 cells), so the design needs a minimum population. At a deadline-met target of 0.98 *no* system is feasible in any cell: the "98% of synchronous invocations within 5× warm latency" service level is out of reach in this setting for VOSS and the baselines alike.

C. RQ3: Does planning attested session state pay?

Table XX compares planners on identical scenarios under medium and high churn. Planning is one of the two largest levers (with lifecycle-aware placement, Fig. 10). The greedy planner lowers sync P99 by 54% (medium churn) and 59% (high churn) against no planning, and by 27% and 57% against a keep-alive TTL; the exact MILP gives 54% and 63% against no planning and 29% and 60% against keep-alive (H1, supported). Planning costs donor memory: the MILP holds about 2.1× the warm memory of keep-alive at medium churn. The gain from the exact solver is small and not established. At medium churn the MILP improves P99 by 1.8% over the greedy planner, below our 5% threshold. At high churn its point estimate is 8.1%, above the threshold, but the 95% interval runs from a 21% reduction to a 1% increase and includes zero (H8, inconclusive): ten seeds can neither confirm that the exact solver matters nor show that it does not. Its measured wall-clock solve takes 4.1 s per cluster (a pool of at most 40 hosts) against 8 ms; the simulated clock charges both the same modelled planning latency of about 0.1 s (Section VIII-A), so the exact planner's results are optimistic about how stale its plans are. The pre-set rule requires the gain to be below 5% in every regime, and the data do not show this at high churn, so we do not claim the rule is met. We keep the greedy planner as the default of all other experiments, a choice made before these runs, and treat the MILP as a reference whose benefit at high churn needs more seeds to settle. What the formulation provides beyond latency is the robust coverage condition, whose value is examined in RQ4.

Fig. 10 removes one component of VOSS at a time (E19, 10% omitting owners). Removing lifecycle-aware placement costs 3.3 s of sync P99, removing planning 1.1 s (and lowers

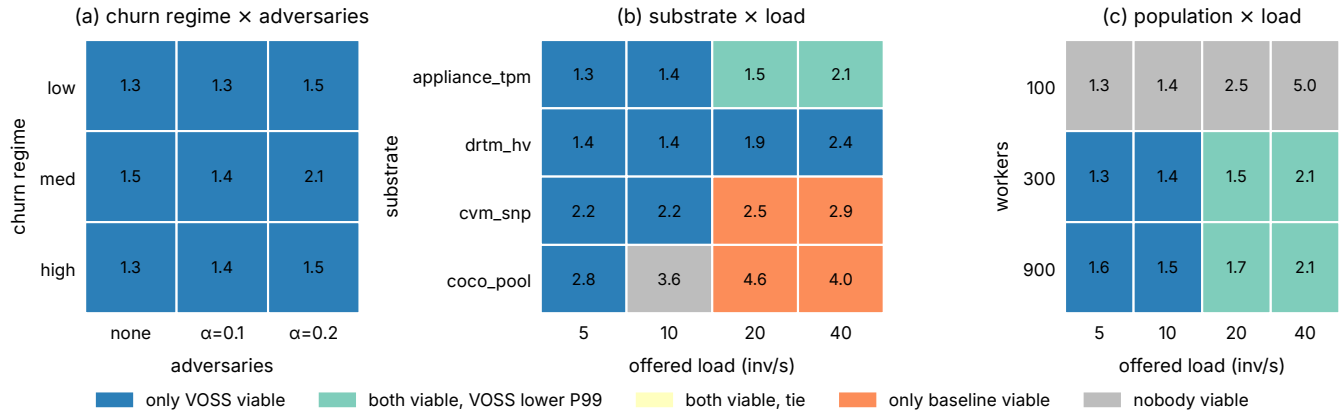


Fig. 9. SLO-feasibility maps. Cell color: which of VOSS and the best simple baseline is feasible (success ≥ 0.99, deadline-met ≥ 0.95); numbers: VOSS sync P99 in seconds. (a) churn regime against adversary fraction, (b) substrate against offered load (inv/s), (c) population against offered load.

TABLE XX
PLANNER COMPARISON (E04): MEAN ± 95% HALF-WIDTH OVER 10 SEEDS. MET: DEADLINE-MET FRACTION; WARM: GB OF IDLE WARM MEMORY.

Regime	Planner	P99 (s)	Met	Warm GB	Exposure	Solve (s)
churn_med	no planning	2.78±1.35	0.940±0.017	1±0	8.5±1.4	0.002±0.000
churn_med	keep-alive TTL	1.78±0.48	0.971±0.018	61±9	7.4±1.1	0.002±0.001
churn_med	greedy planner	1.29±0.09	0.978±0.015	94±14	19.9±1.8	0.008±0.001
churn_med	robust MILP	1.27±0.10	0.979±0.015	129±18	20.1±1.5	4.005±0.505
churn_high	no planning	3.48±1.32	0.923±0.029	1±0	7.5±1.2	0.001±0.000
churn_high	keep-alive TTL	3.28±1.91	0.940±0.030	59±10	7.3±0.9	0.001±0.000
churn_high	greedy planner	1.42±0.17	0.958±0.028	87±13	15.8±1.2	0.008±0.002
churn_high	robust MILP	1.31±0.09	0.958±0.031	119±11	15.2±0.9	4.164±0.650

TABLE XXI
SCHEDULER FAULTS (E15): TIME TO DETECTION, FAILOVER, SUCCESS AND P99 WITHIN THE FAULT WINDOW.

Fault	Detector	Detect (s)	Failover (s)	Success (%)	P99 in window (s)
crash	progress	2.2±0.3	2.7±0.4	99.95±0.02	1.28±0.15
crash	heartbeat_only	5.0±0.0	5.5±0.5	99.87±0.04	1.66±0.33
partition	progress	2.2±0.3	2.6±0.4	99.95±0.02	1.28±0.16
partition	heartbeat_only	5.0±0.0	5.5±0.5	99.87±0.04	1.67±0.31
blackhole	progress	4.2±3.0	4.6±3.0	99.95±0.02	1.28±0.15
blackhole	heartbeat_only	60.0±0.0	59.3±1.1	96.81±0.51	2.90±0.08
slow	progress	13.9±7.9	14.3±8.2	99.95±0.03	1.34±0.14
slow	heartbeat_only	60.0±0.0	59.3±1.1	99.55±0.41	2.00±0.33

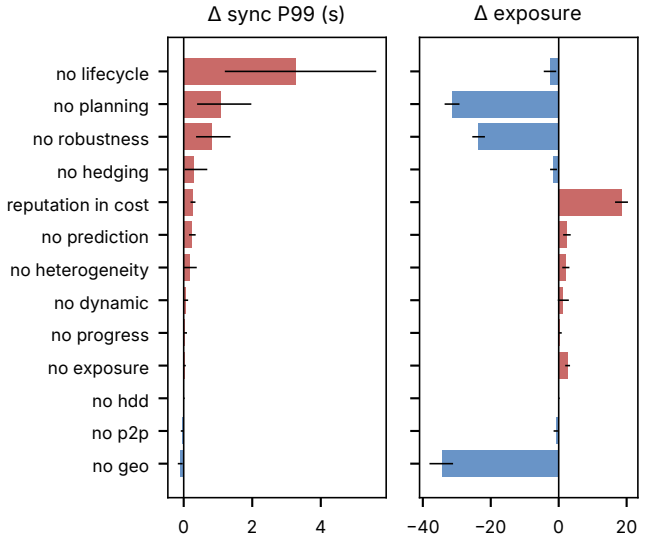


Fig. 10. Ablations (E19): change when a component is removed, relative to full VOSS (paired, 95% intervals). Left: sync P99; right: key exposure.

exposure by 31, because less is pre-warmed and fewer hosts hold keys), removing robust coverage 0.8 s (exposure 24 lower) and removing hedging 0.3 s. The same components that buy P99 (planning, robustness) raise exposure, which is the trade examined in RQ5. Geo-clustering is the only component whose removal *improves* P99 (0.09 s) and cuts exposure (by 34), as Section IX-F discusses.

D. RQ4: Does progress-based detection make malice look like churn?

Scheduler faults. Table XXI injects crashes, partitions, black-holing and slowed scheduler hosts. Heartbeats alone cannot see a black hole: the host keeps reporting liveness while dropping work, so detection waits for the 60 s timeout (60 s) and the run suffers 132 timeouts, against 4.2 s and 0.7 with progress-based detection (H-FT1, supported). For crashes, progress detection needs 2.2 s and failover 2.7 s. A slowed but live scheduler is the hardest case: failover takes 14 s because progress evidence must accumulate before the slowdown is distinguishable from load.

Omitting owners. Table XXII sweeps the fraction α of omitting owners. With $\alpha = 0.2$ VOSS keeps success at 99.95%, whereas heartbeat-only detection with otherwise identical machinery drops to 96.8% (H4, supported) and loses 9× more results to omitting owners. At the same time success under omission stays near the churn-only level, which supports the claim that attested omitting owners behave like churn once detection uses progress. The *robust coverage* constraint is not what keeps success: setting $f = 0$ changes success by only 0.11 percentage points (H2, inconclusive), although it lengthens P99 (Table XXII). Detection and re-dispatch carry success; robust planning buys tail latency.

Hedging. Always hedging cuts P99 but wastes 10.6% extra attempts. Risk-proportional hedging wastes 1.8%, and its P99 is 13% lower at $\alpha = 0.1$, with an interval from -32% to +1% (H-FT3, inconclusive). It is inexpensive insurance, not

TABLE XXII
OMITTING OWNERS (E08), MEAN ± 95% HALF-WIDTH OVER 10 SEEDS. OMISSIONS: RESULTS DROPPED BY ADVERSARIES.

Adversaries	System	Success (%)	P99 (s)	Waste (%)	Omissions
none α=0	VOSS	99.99±0.01	1.27±0.11	1.7±0.5	0±0
none α=0	VOSS, f=0	99.99±0.01	1.60±0.24	2.2±0.6	0±0
none α=0	heartbeat-only	99.98±0.01	1.36±0.15	2.0±0.7	0±0
mixed α=0.1	VOSS	99.95±0.04	1.32±0.12	2.2±0.5	38±14
mixed α=0.1	VOSS, f=0	99.96±0.03	2.15±0.60	2.9±0.6	37±11
mixed α=0.1	heartbeat-only	98.19±0.75	1.36±0.13	4.0±0.9	298±115
mixed α=0.2	VOSS	99.95±0.03	1.43±0.16	2.6±0.5	61±13
mixed α=0.2	VOSS, f=0	99.83±0.17	2.05±0.52	3.5±0.8	80±31
mixed α=0.2	heartbeat-only	96.79±1.13	1.39±0.16	5.6±1.3	527±161
all α=0.2	VOSS	99.94±0.05	1.37±0.10	2.3±0.7	31±9
all α=0.2	VOSS, f=0	99.96±0.03	2.28±1.00	2.7±0.5	28±8
all α=0.2	heartbeat-only	97.70±1.26	1.42±0.14	5.2±2.4	385±227

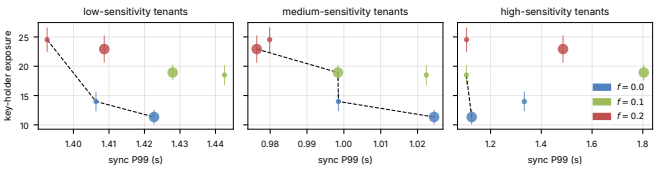


Fig. 11. Latency–exposure–robustness (E23). Each point is a (f, exposure weight) configuration (marker size: weight) with 95% error bars on exposure; the dashed line is the Pareto front per tenant sensitivity class.

demonstrated to be necessary.

E. RQ5: Robustness, latency and exposure

Robustness f and the exposure weight trade latency against key exposure (E23, Fig. 11). The aggregate front has 3 points. Exposure grows with f (13, 19 and 24 at $f = 0$, 0.1, 0.2), consistent with covering any f failures requiring more key-holding hosts. Aggregate sync P99 stays within 1.27–1.35 s across configurations, so the knee at $f = 0.1$ saves 25% of exposure against the best-latency front point for 1.6% more P99 (H3, supported). Per tenant class the picture is less benign: low-sensitivity tenants stay at about 1.4 s or below, but for high-sensitivity tenants P99 reaches 1.8 s in one robust configuration, while an $f = 0$ configuration has the lowest exposure at nearly the best P99. The aggregate knee therefore should not be applied to every tenant class; each needs its own operating point.

F. RQ6: Components that do not earn their complexity

Geo-clustering. Table XXIII compares a single global scheduler against static regions, static k-means and dynamic clusters on 600 workers (about 225 of them attested) at 20 invocations per second, with clusters of roughly 20 to 35 attested hosts (target 30). The global scheduler plans over at most 40 hosts, the planning-pool cap of Section VI-D, which is part of why it holds fewer keys, and nothing in this load stresses one scheduler's decision capacity. Dynamic clusters have *higher* P99 than the global scheduler by 17% (Europe), 17% (EU+NA) and 5% (world) (H7, refuted at all three populations), and hold 5.1× the key exposure in Europe, since each of the 9 clusters must cover demand with its own key holders. The network share of latency falls in some populations (Table XXIII), but not enough to offset the cost of splitting the pool. We attribute this to fewer hosts per cluster for pooling warm state and keys, but did not isolate the mechanism. We found no population in which geo-clustering wins at the scale we simulate.

TABLE XXIII
GEO-CLUSTERING (E05): MEAN ± 95% HALF-WIDTH OVER 10 SEEDS.

Population	Clustering	P99 (s)	Network (s)	Clusters	Exposure
Europe	one global scheduler	1.06±0.09	0.092±0.015	1.0±0.0	12±1
Europe	static regions	1.06±0.09	0.092±0.015	1.0±0.0	12±1
Europe	static k-means	1.25±0.09	0.124±0.031	10.4±0.5	63±4
Europe	dynamic	1.24±0.10	0.127±0.037	9.1±0.6	61±4
EU + NA	one global scheduler	1.14±0.09	0.154±0.021	1.0±0.0	14±1
EU + NA	static regions	1.20±0.13	0.123±0.014	2.0±0.0	24±2
EU + NA	static k-means	1.31±0.13	0.151±0.031	7.5±0.4	54±3
EU + NA	dynamic	1.33±0.20	0.127±0.013	6.4±0.6	52±4
world	one global scheduler	1.21±0.11	0.192±0.016	1.0±0.0	13±1
world	static regions	1.36±0.12	0.167±0.013	6.6±0.3	43±4
world	static k-means	1.31±0.11	0.164±0.020	7.0±0.5	48±3
world	dynamic	1.27±0.11	0.157±0.018	6.0±0.3	46±3

TABLE XXIV
SIMPLE BASELINE VERSUS VOSS (E10).

Hardware	System	P99 (s)	Met	Exposure
homogeneous	simple baseline	3.90±1.00	0.962±0.012	64±4
homogeneous	VOSS	2.38±0.47	0.978±0.007	122±7
mixed	simple baseline	2.66±0.24	0.969±0.012	54±3
mixed	VOSS	1.31±0.09	0.989±0.003	95±4

Routing and caching. Without flash crowds, nearest-region routing meets 0.991 of deadlines (P99 1.20 s), whereas routing to the least-loaded cluster meets 0.911 (P99 1.39 s) because it ignores distance. For caching (E09), cost-aware GreedyDual with peer-to-peer fetch and an HDD tier is at most 3% better than LRU in sync P99 and the intervals include zero (H6, inconclusive); all variants start about 98% of invocations from a warm VM, which leaves little for the disk tiers to improve.

Reputation and accounting. Attested receipts are off the critical path (H10, supported): lengthening the settlement period from 10 to 300 s or the batch from 0.2 to 5 s changes P99 by no more than ±1.5%. The stream is small, about 17 rows/s and 4.5 KB/s at 20 inv/s offered load; every receipt is accounted for (0 unexplained, at most 1 missing at the end of a run). Reputation adds little: using a Beta-decay score as a tie-breaker lowers P99 by 2.4% (weak), and credit is highly concentrated (Gini 0.93; the top 10% of workers earn 93% of credit and 65% earn almost none), a fairness problem for an incentive that relies on credits.

G. RQ7: Baselines, commercial reference and scale

Simple baseline. Table XXIV compares VOSS with the simple baseline on a homogeneous, stable, benign pool, where we expected the baseline to match. It does not: VOSS has 64% lower P99 (H11, refuted), which we attribute to planning's pre-warming against churn. The baseline, however, holds about half of VOSS's key exposure (VOSS is 1.9× higher), consistent with its placing keys on fewer hosts. Whether the latency is worth the exposure is a policy choice that the Pareto front of RQ5 lets a deployer make.

Commercial reference. Table XXV compares VOSS with commercial-style references on the same workload and engine at list prices. At 20 inv/s VOSS's P99 is 14% and its P50 45% above the Lambda-like reference. Cost per million invocations,

TABLE XXV
COMMERCIAL-STYLE REFERENCES VERSUS VOSS (E17). MET: DEADLINE-MET FRACTION; COST IN USD PER MILLION INVOCATIONS.

Load (inv/s)	System	P50 (s)	P99 (s)	Met	USD/M inv.
10	VOSS	0.425±0.083	1.33±0.08	0.987±0.005	19.6±0.9
10	Lambda-like	0.292±0.090	1.21±0.12	0.999±0.000	12.1±4.5
10	Azure-like	0.292±0.090	1.33±0.13	0.997±0.000	12.0±4.4
20	VOSS	0.425±0.082	1.27±0.11	0.987±0.006	9.8±0.5
20	Lambda-like	0.292±0.090	1.11±0.10	0.999±0.000	12.1±4.6
20	Azure-like	0.292±0.090	1.17±0.11	0.998±0.000	11.9±4.4
40	VOSS	0.442±0.084	1.50±0.23	0.979±0.009	4.9±0.2
40	Lambda-like	0.292±0.091	1.10±0.10	0.999±0.000	12.1±4.6
40	Azure-like	0.292±0.090	1.15±0.11	0.998±0.000	11.9±4.5

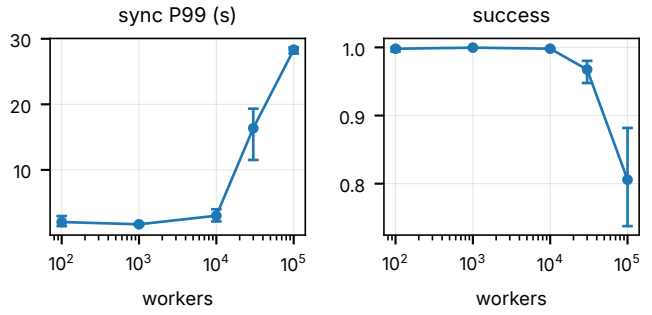


Fig. 12. Scalability (E18): sync P99 and success against population (log scale), mean with 95% intervals; the two largest points use 3 seeds.

including the control plane but excluding volunteer credits, falls from 20 USD at 10 inv/s through 10 to 5 at 40 inv/s as the fixed tier is amortized, against about 12 USD for the references. VOSS is cost-competitive only above roughly 20 inv/s on this workload and never has lower latency.

Scalability. Fig. 12 and Table XXVI scale the population from 10^2 to 10^5 workers at constant load per worker (0.01 inv/s) in a cheaper "scalability mode" (coarse heartbeats, greedy planning, 2 simulated minutes). Up to 10^4 workers success is 99.8%. At 3×10^4 it falls to 97% with a decision P99 of 14 s, and at 10^5 to 81% with sync P99 28 s and decision P99 30 s: decision latency grows from about 0.1 s to tens of seconds. Per-worker control traffic stays flat (0.70–1.08 messages per second per worker), so the limit lies in the central decision path rather than the message volume. We did not isolate which orchestrator or router component saturates first, and the two largest points use three seeds.

H. Sensitivity to calibration

We repeated the key comparison under the placeholder and the laptop-measured bundles (Table XXVII). VOSS's P99 is 1.17 s with measured parameters against 1.32 s with placeholders, while the simple baseline moves from 3.9 s to 4.6 s, so the advantage of planning grows under the measured parameters. The main qualitative conclusion is not an artifact of the placeholders; absolute values should not be read as predictions.

I. Hypothesis summary

Of the hypotheses stated before the runs, 11 are supported, 2 weak, 6 inconclusive and 5 refuted (Table XXVIII). The refuted entries (geo-clustering, the simple baseline on homogeneous hosts, and the cost ratio of confidential VMs) and the inconclusive ones (the robust constraint for success, the MILP over greedy planning, caching and hedging) are as informative as the supported ones.

TABLE XXVI
SCALABILITY (E18): ORCHESTRATOR AND CONTROL-PLANE LOAD.

Workers	Seeds	Success (%)	P99 (s)	Decide P99 (s)	Msg/s/worker	Orch. CPU (s/s)
100	10	99.8±0.3	2.1±0.8	0.10±0.01	0.97±0.04	0.000±0.000
1 000	10	100.0±0.1	1.7±0.2	0.10±0.01	0.74±0.01	0.002±0.000
10 000	10	99.8±0.1	3.0±0.9	0.48±0.59	0.73±0.01	0.021±0.000
30 000	3	96.8±1.6	16.4±3.9	14.45±11.95	0.74±0.01	0.062±0.000
100 000	3	80.6±7.2	28.3±0.5	29.83±8.05	0.82±0.03	0.207±0.001

TABLE XXVII
CALIBRATION SENSITIVITY (E24): PLACEHOLDER VERSUS LAPTOP-MEASURED PARAMETERS.

Bundle	System	P50 (s)	P99 (s)	Met	Exposure
placeholder (C0 design)	VOSS	0.429±0.083	1.32±0.11	0.984±0.006	47±4
placeholder (C0 design)	simple baseline	0.509±0.095	3.89±0.60	0.965±0.010	53±3
measured (laptop)	VOSS	0.239±0.040	1.17±0.25	0.978±0.005	37±4
measured (laptop)	simple baseline	0.266±0.052	4.59±0.65	0.954±0.011	53±3

X. DISCUSSION

What the evidence supports. The central design claim holds in simulation: managing attested session state as a planned, perishable resource beats reactive policies. Planning cuts tail latency by roughly half against no planning under churn, lifecycle-aware placement and planning are the two largest contributors in the ablation, and progress-based detection is what keeps success high when owners omit. These statements concern mechanisms and are supported by paired intervals; they hold across churn regimes and adversary fractions in the feasibility maps rather than at one operating point.

What it does not. Four expected benefits did not materialize. First, geo-clustering is slower than a single scheduler and holds several times the exposure at our scale. Second, the exact MILP is not demonstrably better than the greedy planner (its point estimate at high churn is 8.1%, but the interval includes zero), so its cost is justified only by the coverage guarantee, which we could not show matters for *success*. Third, caching sophistication and reputation tie-breaking change P99 by a few percent at most, and credit under the current rules is extremely concentrated. Fourth, a commercial-style reference has lower latency, and VOSS is cheaper per invocation only once the fixed control plane is amortized. None of this contradicts the premise that volunteer hardware can serve latency-sensitive functions at some SLO; it narrows which components earn their complexity. A leaner VOSS (attestation, a greedy robust planner, a cost-based placer with hedging, progress-based detection and epoch-fenced failover, plus receipts) is the version our evidence favors.

Exposure is the price of latency. Planning, robustness and warm state all enlarge the set of hosts holding a tenant's key. The simple baseline has about half of VOSS's exposure, and the Pareto front shows that moderate robustness captures most of the benefit for aggregate latency but not for every tenant class. A deployment should expose exposure and robustness as per-tenant policy, not as global constants.

TABLE XXVIII
HYPOTHESIS SUMMARY. Δ REL.: PAIRED RELATIVE CHANGE WITH 95% BOOTSTRAP INTERVAL; VERDICT BY THE RULES OF SECTION VIII-C. "–": COST-SIDE INFORMATION, NO VERDICT.

H	Claim	Exp.	Cell	Δ rel. [95% CI]	Verdict
H1	MILP planner cuts sync P99 vs no planning	E04	churn_med	-54.5% [-105.8, -12.6]	**supported**
H1	MILP planner cuts sync P99 vs no planning	E04	churn_high	-62.5% [-100.3, -27.0]	**supported**
H1	... and vs keep-alive TTL	E04	churn_med	-28.7% [-57.6, -7.7]	**supported**
H1	... and vs keep-alive TTL	E04	churn_high	-60.2% [-124.6, -12.2]	**supported**
H1	donor memory held warm: MILP vs keep-alive TTL (cost side)	E04	churn_med	+109.8% [+90.2, +125.8]	–
H2	f-robust plans keep success where non-robust collapse (α=0.2 mixed)	E08	mixed α=0.2	-0.1% [-0.3, +0.0]	inconclusive
H3	Pareto front has a knee; moderate exposure loses little latency	E23		+1.6%	**supported**
H4	progress detection beats heartbeat-only on success under α=0.2	E08	mixed α=0.2	-3.2% [-4.3, -2.1]	**supported**
H5	appliance warm start $\approx$ Firecracker without attestation (L3)	E-CS1 (sim)		-2.0%	**supported**
H5	appliance snapshot start $\approx$ Firecracker without attestation (L2)	E-CS1 (sim)		+2.5%	**supported**
H5	CVM classes are 10-100x slower than the appliance to establish trust	E-CS1 (sim)		-92.7%	**refuted**
H6	GDSF + P2P + HDD tier beat LRU without P2P/HDD (sync P99)	E09		+2.8% [-2.9, +11.9]	inconclusive
H6	... GDSF vs LRU alone	E09		+1.7% [-0.2, +4.1]	inconclusive
H7	geo clusters reduce sync P99 vs one global scheduler (EU)	E05	regions=[eu]	+17.3% [+12.5, +22.0]	**refuted**
H7	... EU+NA	E05	regions=[eu, na]	+16.6% [+5.2, +33.1]	**refuted**
H7	... world	E05	regions=[]	+4.5% [+0.9, +8.7]	**refuted**
H8	planner MILP beats greedy by $\geq$5% sync P99 (high churn)	E04	churn_high	-8.1% [-20.8, +1.1]	inconclusive
H9	distribution-aware (quantile) beats expectation-only on P99 at high churn	E03	churn_high, name=blr	-2.8% [-5.3, -0.8]	weak
H10	accounting is off the critical path: settlement period 10 s vs 300 s	E13	faults=[]	+1.5% [-0.2, +4.2]	**supported**
H10	... receipt batch 0.2 s vs 5 s	E13	faults=[]	-1.0% [-3.3, +1.0]	**supported**
H11	simple baseline matches VOSS when homogeneous, stable and benign	E10	hw=C1	+63.7% [+25.0, +112.8]	**refuted**
H-FT1	progress detection vs heartbeat-only: success with a black-holed scheduler	E15	faults=sched_blackhole	+3.2% [+2.7, +3.7]	**supported**
H-FT3	risk-proportional hedging (θ=.3, 5%) cuts sync P99 under α=0.1	E16	mixed α=0.1	-13.3% [-32.5, +1.4]	inconclusive
H-FT3	... at what extra-attempt cost	E16	mixed α=0.1	+253.0% [+182.6, +332.0]	–
RQ15	VOSS vs Lambda-like reference, sync P99 at 20 inv/s	E17	rate=20	+14.1% [+10.4, +18.3]	–
E12	reputation tie-break (beta-decay) improves sync P99 vs no reputation	E12		-2.4% [-4.4, -0.8]	weak
E12	... and success rate	E12		-0.0% [-0.0, +0.0]	inconclusive

Substrate and population matter more than tuning. The feasibility maps show that the choice of trust substrate and a minimum population dominate parameter tuning: on confidential-VM substrates the simple baseline can suffice at high load, and below a few hundred hosts no system is feasible.

Reliance on attestation. Every robustness result uses omission and timing faults only. If attestation failed, for example through the physical attacks excluded in Section V-A, adversaries could forge results and read keys, and exposure would no longer be a proxy for risk. This is why the design treats liveness and owner notices as hints and verifies progress.

A. Extension to function workflows

The model of Sections IV–VII treats invocations as independent. Many serverless applications are instead *workflows*, directed acyclic graphs (DAGs) of functions in which the completion of one function triggers its successors [40]. This is outside the scheduling contribution of this paper and we do not evaluate it, but the structure is exactly the kind of information a trust-aware scheduler can use: when a function is called, the functions in its subtree will be called soon, so their start-up and provisioning can be overlapped with the parent's execution instead of paid in sequence. Without that overlap every stage of a chain can pay a cold start in turn, a *cascading cold start* [41]. Table XXIX states how each part of VOSS would extend, and which prior ideas apply.

Two properties make workflows a natural fit. First, lead time gives the planner slack: the provisioning round trip and a snapshot restore of a child can be hidden behind the parent, which converts a latency that the fast loop must otherwise price into an overlap. Second, keeping workflow state at the router rather than on a volunteer host follows the progress principle of Section IV and keeps volunteers out of the coordination path. The risk is speculation: pre-warming a child that a conditional branch does not call wastes donor memory, and warming it on an additional host enlarges exposure, so look-ahead warming should be budgeted like hedging and restricted to hosts that are already provisioned for the tenant. Evaluating this extension needs a workflow benchmark such as SeBS-Flow [40] and is future work.

B. Threats to validity

- **Simulation, partly validated.** No multi-machine deployment has validated the simulator end to end. A single-box

TABLE XXIX
HOW VOSS EXTENDS FROM INDEPENDENT INVOCATIONS TO FUNCTION WORKFLOWS (DAGS). NOT EVALUATED IN THIS PAPER.

Concern	Independent invocations	Workflow extension
Demand forecast	per function, from its own arrivals	*derived demand* $\lambda_f = \sum_p P(p \to f)\,\lambda_p$ over DAG edges; edge probabilities are fixed for static DAGs and learned online for conditional branches; the hot set grows to include the children of hot functions
Warm state	planner pins tiers L2–L3 by value density	weight the value of warming a child by its *lead time*, the parent's predicted remaining time; on `DISPATCH` of a parent, issue `WARM` for its children so start-up overlaps execution (look-ahead pre-warming, as in [41], [42])
Placement	cost includes client-to-host network	add an edge term for moving the parent's output; prefer the same host or cluster for parent and child, or one sandbox per workflow where isolation allows [43], [44]; choose the data-passing path per edge [45]
Trust and exposure	key provisioned per tenant	unchanged within one tenant, since all functions of a workflow share y_{tj} and no new key holder appears; a cross-tenant workflow needs both tenants' keys and its exposure is the union; intermediate data are tenant data and stay on attested channels
Fault tolerance	router keeps a pending copy; hedge and retry per invocation	keep workflow state at the trusted router, with per-node attested receipts as the progress evidence; a failed node is re-executed idempotently without re-running completed nodes; hedging applies per node under one budget for the DAG
Accounting	one receipt per invocation	one receipt and credit per node

emulation (E01, Section VIII-B) agrees on success rate and start-up levels but not on the shape of the latency distribution, and it did not exercise the network, attestation, key release, the session boot or failover. Policies are the real code, but the world model (network, churn, adversaries) is a model. We interpret differences between designs, not absolute latencies.

- **Calibration scope.** Calibration comes from one consumer laptop with a hard disk, and the appliance ran in host mode. Confidential-VM and DRTM parameters are literature values. E24 shows the main comparison survives a change of bundle, but a different CPU, disk or firmware could shift absolute costs.
- **Assumed workload and churn.** No public trace of consumer donation sessions exists; churn shapes are assumed from availability studies and swept. The function catalogue follows SeBS but its sizes are assumed. It has 12 functions of six tenants, so the hot-set limits of the planner never bind and the long tail of rarely used functions that production platforms see is not exercised; functions make no calls to external services; and 11 of the 12 are marked idempotent by us, an assumption on which retries, hedging and the success results under omission rest (non-idempotent functions are not retried). The adversary model is omission and timing with a fixed menu of strategies; an adaptive adversary that learns the scheduler could do worse.
- **The scheduler's knowledge of costs is idealised.** The scheduler uses calibrated nominal start-up, trust and network times, and takes declared host speeds and link rates as exact (Table IX), whereas the simulated world adds jitter and noise. A deployment would estimate these online. We did not sweep prediction error or staleness, so the results show the benefit of the *policies* under this idealisation, not their robustness to cost misestimation.
- **Network and decision costs.** Messages are delayed by a geographic model plus 20 ms for each endpoint behind a relay (15% of workers by default); NAT traversal, relay capacity, connection setup, address changes and reconnection after a scheduler change are not modelled, and the paper does not specify how a worker reaches a scheduler that is itself a volunteer host. Decision latency is charged by a placeholder formula (about 0.1 s for a plan of the sizes used here, for the greedy and the exact planner alike), whereas the exact planner's measured solve took 4.1 s; the exact-planner results are therefore optimistic about plan staleness.
- **Scale.** The two largest scalability points use three seeds and a cheaper simulator mode, and a 10^5-worker deployment may behave differently. The geo-clustering comparison (600 workers, about 225 attested, 20 inv/s) is too small to stress a single scheduler, and the global baseline plans over at most 40 hosts.
- **Statistics and selection.** Ten seeds give wide intervals for tail metrics, and several comparisons are inconclusive rather than null. We ran many experiments, so a verdict on a single comparison should be read with that in mind. Specifications were frozen before the final sweep, but pilots informed which cells to include.
- **Literature parameters.** Parameters taken from cited studies (for example confidential-VM start-up) are quoted as reported and tagged for re-verification.

C. Future work

Validate on a small real testbed with end-to-end measurements, which would also test the host-mode appliance assumption; measure confidential-VM hosts to replace literature parameters; shard the orchestrator tier and re-measure scalability; revisit geo-clustering at larger populations and with shared trust across clusters; evaluate workflow-aware look-ahead warming (Section X-A); and study adaptive adversaries and credit-based incentives given the observed credit concentration.

XI. CONCLUSION

Latency-sensitive serverless computing on volunteer machines needs more than redundancy and keep-alive: churn destroys the state that makes short functions fast, and untrusted owners make that state dangerous to create. We showed that attestation of a locked appliance reduces malicious owners to omission and timing failures, which exposes the real scheduling problem, the management of attested session state that is costly to establish, perishable and exposing. VOSS addresses it with a slow robust provisioning planner and a fast cost-based placer, progress-based detection of gray failures, and lease and epoch-fenced scheduler failover without volunteer consensus. In a calibrated simulator it is feasible in regimes where simple

baselines are not (9 of 9 and 13 of 16 cells at a 3 s sync P99 target), lowers tail latency by 54–59% against no planning, and keeps success at 99.95% with 20% omitting owners. The advantage is not universal: on confidential-VM substrates at high load the simple baseline is the only feasible system, and geo-clustering, caching sophistication and reputation do not pay for themselves in our experiments, while the benefit of the exact solver over greedy planning is not established. The next step is a real multi-machine testbed to validate the simulator, which this paper does not yet provide.

REFERENCES

[1] M. Shahrad, R. Fonseca, I. Goiri, G. Chaudhry, P. Batum, J. Cooke, E. Laureano, C. Tresness, M. Russinovich, and R. Bianchini, "Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider," in *Proc. USENIX ATC*, 2020, pp. 205–218.

[2] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa, "Firecracker: Lightweight virtualization for serverless applications," in *Proc. USENIX NSDI*, 2020, pp. 419–434.

[3] E. Oakes, L. Yang, D. Zhou, K. Houck, T. Harter, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "SOCK: Rapid task provisioning with serverless-optimized containers," in *Proc. USENIX ATC*, 2018, pp. 57–70.

[4] L. Ao, G. Porter, and G. M. Voelker, "FaaSnap: FaaS made fast using snapshot-based VMs," in *Proc. EuroSys*, 2022, pp. 730–746.

[5] A. Fuerst and P. Sharma, "FaaSCache: Keeping serverless computing alive with greedy-dual caching," in *Proc. ACM ASPLOS*, 2021, pp. 386–400.

[6] D. P. Anderson, J. Cobb, E. Korpela, M. Lebofsky, and D. Werthimer, "SETI@home: An experiment in public-resource computing," *Commun. ACM*, vol. 45, no. 11, pp. 56–61, 2002.

[7] D. P. Anderson, "BOINC: A system for public-resource computing and storage," in *Proc. IEEE/ACM Int. Workshop on Grid Computing*, 2004, pp. 4–10.

[8] S. Arnautov, B. Trach, F. Gregor, T. Knauth, A. Martin, C. Priebe, J. Lind, D. Muthukumaran, D. O'Keeffe, M. L. Stillwell, D. Goltzsche, D. Eyers, R. Kapitza, P. Pietzuch, and C. Fetzer, "SCONE: Secure Linux containers with Intel SGX," in *Proc. USENIX OSDI*, 2016, pp. 689–703, tODO verify.

[9] "Confidential serverless computing (Wallet)," arXiv:2504.21518; USENIX NSDI 2026, 2025, tODO verify authors and final title.

[10] P. Huang, C. Guo, L. Zhou, J. R. Lorch, Y. Dang, M. Chintalapati, and R. Yao, "Gray failure: The achilles' heel of cloud-scale systems," in *Proc. Workshop on Hot Topics in Operating Systems (HotOS)*, 2017, pp. 150–155, tODO verify.

[11] F. Cristian, "Understanding fault-tolerant distributed systems," *Commun. ACM*, vol. 34, no. 2, pp. 56–78, 1991, tODO verify.

[12] A. Avižienis, J.-C. Laprie, B. Randell, and C. Landwehr, "Basic concepts and taxonomy of dependable and secure computing," *IEEE Trans. Dependable and Secure Computing*, vol. 1, no. 1, pp. 11–33, 2004, tODO verify.

[13] D. Levin, J. R. Douceur, J. R. Lorch, and T. Moscibroda, "TrInc: Small trusted hardware for large distributed systems," in *Proc. USENIX NSDI*, 2009, pp. 1–14.

[14] G. S. Veronese, M. Correia, A. N. Bessani, L. C. Lung, and P. Verissimo, "Efficient Byzantine fault-tolerance," *IEEE Trans. Computers*, vol. 62, no. 1, pp. 16–30, 2013.

[15] J. Cadden, T. Unger, Y. Awad, H. Dong, O. Krieger, and J. Appavoo, "SEUSS: Skip redundant paths to make serverless fast," in *Proc. EuroSys*, 2020, pp. 32:1–32:15.

[16] H. Birkholz, D. Thaler, M. Richardson, N. Smith, and W. Pan, "Remote ATtestation procedureS (RATS) architecture," IETF, RFC 9334, 2023, tODO verify.

[17] Trusted Computing Group, "Trusted platform module library specification, family 2.0," TCG specification, 2019, tODO verify revision.

[18] N. Schear, P. T. Cable, T. M. Moyer, B. Richard, and R. Rudd, "Bootstrapping and maintaining trust in the cloud," in *Proc. Annual Computer Security Applications Conf. (ACSAC)*, 2016, pp. 65–77.

[19] B. Javadi, D. Kondo, A. Iosup, and D. Epema, "The Failure Trace Archive: Enabling the comparison of failure measurements and models of distributed systems," *J. Parallel and Distributed Computing*, vol. 73, no. 8, pp. 1208–1223, 2013.

[20] N. Hayashibara, X. Défago, R. Yared, and T. Katayama, "The φ accrual failure detector," in *Proc. IEEE Symp. on Reliable Distributed Systems (SRDS)*, 2004, pp. 66–78.

[21] K. Ousterhout, P. Wendell, M. Zaharia, and I. Stoica, "Sparrow: Distributed, low latency scheduling," in *Proc. ACM SOSP*, 2013, pp. 69–84.

[22] Y. Zhang, Í. Goiri, G. I. Chaudhry, R. Fonseca, S. Elnikety, C. Delimitrou, and R. Bianchini, "Faster and cheaper serverless computing on harvested resources," in *Proc. ACM SOSP*, 2021, pp. 724–739.

[23] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, "Large-scale cluster management at Google with Borg," in *Proc. EuroSys*, 2015, pp. 18:1–18:17.

[24] T. Rausch, A. Rashed, and S. Dustdar, "Optimized container scheduling for data-intensive serverless edge computing," *Future Generation Computer Systems*, vol. 114, pp. 259–271, 2021, tODO verify.

[25] V. Costan and S. Devadas, "Intel SGX explained," IACR Cryptology ePrint Archive, 2016/086, 2016.

[26] AMD, "SEV-SNP: Strengthening VM isolation with integrity protection and more," White paper, 2020.

[27] C. Segarra, T. Pfandzelter *et al.*, "Serverless confidential containers: Challenges and opportunities," in *Proc. Workshop on Serverless Systems, Applications and Methodology (SESAME)*, 2024, tODO verify authors.

[28] M. Misono, D. Stavrakakis, N. Santos, and P. Bhatotia, "Confidential VMs explained: An empirical analysis of AMD SEV-SNP and Intel TDX," in *Proc. ACM SIGMETRICS*, 2025, tODO verify.

[29] J. Dean and L. A. Barroso, "The tail at scale," *Commun. ACM*, vol. 56, no. 2, pp. 74–80, 2013.

[30] D. Bertsimas and M. Sim, "The price of robustness," *Operations Research*, vol. 52, no. 1, pp. 35–53, 2004.

[31] H. N. Jacob, C. Werling, R. Buhren, and J.-P. Seifert, "faulTPM: Exposing AMD fTPMs' deepest secrets," in *Proc. IEEE European Symp. on Security and Privacy (EuroS&P)*, 2023, tODO verify.

[32] J. R. Douceur, "The Sybil attack," in *Proc. Int. Workshop on Peer-to-Peer Systems (IPTPS)*, 2002, pp. 251–260.

[33] B. Holmes, J. Waterman, and D. Williams, "SEVeriFast: Minimizing the root of trust for fast startup of SEV microvms," in *Proc. ACM ASPLOS*, 2024, tODO verify title and pages.

[34] D. Li, Z. Mi, C. Ji, Y. Tan, B. Zang, H. Guan, and H. Chen, "Bifrost: Analysis and optimization of network I/O tax in confidential virtual machines," in *Proc. USENIX ATC*, 2023, tODO verify.

[35] Q. Huangfu and J. A. J. Hall, "Parallelizing the dual revised simplex method," *Mathematical Programming Computation*, vol. 10, no. 1, pp. 119–142, 2018.

[36] G. Shafer and V. Vovk, "A tutorial on conformal prediction," *J. Machine Learning Research*, vol. 9, pp. 371–421, 2008.

[37] M. Burrows, "The Chubby lock service for loosely-coupled distributed systems," in *Proc. USENIX OSDI*, 2006, pp. 335–350, tODO verify.

[38] M. Copik, G. Kwasniewski, M. Besta, M. Podstawski, and T. Hoefler, "SeBS: A serverless benchmark suite for function-as-a-service computing," in *Proc. ACM/IFIP Middleware*, 2021, pp. 64–78.

[39] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[40] M. Copik, M. Chrapek, L. Schmid, A. Calotoiu, and T. Hoefler, "SeBS-Flow: Benchmarking serverless cloud function workflows," arXiv preprint, 2024, tODO verify authors, venue and year.

[41] N. Daw, U. Bellur, and P. Kulkarni, "Xanadu: Mitigating cascading cold starts in serverless function chain deployments," in *Proc. ACM/IFIP Middleware*, 2020, pp. 356–370, tODO verify.

[42] A. Mahgoub, E. B. Yi, K. Shankar, S. Elnikety, S. Chaterji, and S. Bagchi, "ORION and the three rights: Sizing, bundling, and prewarming for serverless DAGs," in *Proc. USENIX OSDI*, 2022, pp. 303–320, tODO verify.

[43] I. E. Akkus, R. Chen, I. Rimac, M. Stein, K. Satzke, A. Beck, P. Aditya, and V. Hilt, "SAND: Towards high-performance serverless computing," in *Proc. USENIX ATC*, 2018, pp. 923–935, tODO verify.

[44] S. Kotni, A. Nayak, V. Ganapathy, and U. Bellur, "Faastlane: Accelerating function-as-a-service workflows," in *Proc. USENIX ATC*, 2021, pp. 805–820, tODO verify.

[45] A. Mahgoub, K. Shankar, S. Mitra, A. Klimovic, S. Chaterji, and S. Bagchi, "SONIC: Application-aware data passing for chained serverless applications," in *Proc. USENIX ATC*, 2021, pp. 285–301, tODO verify.